

5 The Many Variables & The Spurious Waffles

One of the most reliable sources of waffles in North America, if not the entire world, is a Waffle House diner. Waffle House is nearly always open, even just after a hurricane. Most diners invest in disaster preparedness, including having their own electrical generators. As a consequence, the United States' disaster relief agency (FEMA) informally uses Waffle House as an index of disaster severity.⁷⁹ If the Waffle House is closed, that's a serious event.

It is ironic then that steadfast Waffle House is associated with the nation's highest divorce rates ([FIGURE 5.1](#)). States with many Waffle Houses per person, like Georgia and Alabama, also have some of the highest divorce rates in the United States. The lowest divorce rates are found where there are zero Waffle Houses. Could always-available waffles and hash brown potatoes put marriage at risk?

Probably not. This is an example of a misleading correlation. No one thinks there is any plausible mechanism by which Waffle House diners make divorce more likely. Instead, when we see a correlation of this kind, we immediately start asking about other variables that are really driving the relationship between waffles and divorce. In this case, Waffle House began in Georgia in the year 1955. Over time, the diners spread across the Southern United States, remaining largely within it. So Waffle House is associated with the South. Divorce is not a uniquely Southern institution, but the Southern United States has some of the highest divorce rates in the nation. So it's probably just an accident of history that Waffle House and high divorce rates both occur in the South.

Such accidents are commonplace. It is not surprising that Waffle House is correlated with divorce, because correlation in general is not surprising. In large data sets, every pair of variables has a statistically discernible non-zero correlation.⁸⁰ But since most correlations do not indicate causal relationships, we need tools for distinguishing mere association from evidence of causation. This is why so much effort is devoted to [MULTIPLE REGRESSION](#), using more than one predictor variable to simultaneously model an outcome. Reasons given for multiple regression models include:

- (1) Statistical “control” for confounds. A *confound* is something that misleads us about a causal influence—there will be a more precise definition in the next chapter. The spurious waffles and divorce correlation is one type of confound, where southernness makes a variable with no real importance (Waffle House density) appear to be important. But confounds are diverse. They can hide important effects just as easily as they can produce false ones.
- (2) Multiple and complex causation. A phenomenon may arise from multiple simultaneous causes, and causes can cascade in complex ways. And since one cause can hide another, they must be measured simultaneously.

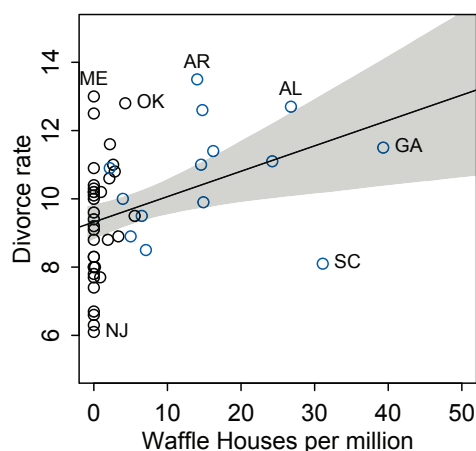


FIGURE 5.1. The number of Waffle House diners per million people is associated with divorce rate (in the year 2009) within the United States. Each point is a State. “Southern” (former Confederate) States shown in blue. Shaded region is 89% percentile interval of the mean. These data are in `data(WaffleDivorce)` in the `rethinking` package.

- (3) Interactions. The importance of one variable may depend upon another. For example, plants benefit from both light and water. But in the absence of either, the other is no benefit at all. Such **INTERACTIONS** occur very often. Effective inference about one variable will often depend upon consideration of others.

In this chapter, we begin to deal with the first of these two, using multiple regression to deal with simple confounds and to take multiple measurements of association. You’ll see how to include any arbitrary number of *main effects* in your linear model of the Gaussian mean. These main effects are additive combinations of variables, the simplest type of multiple variable model. We’ll focus on two valuable things these models can help us with: (1) revealing *spurious* correlations like the Waffle House correlation with divorce and (2) revealing important correlations that may be *masked* by unrevealed correlations with other variables. Along the way, you’ll meet **CATEGORICAL VARIABLES**, which require special handling compared to continuous variables.

However, multiple regression can be worse than useless, if we don’t know how to use it. Just adding variables to a model can do a lot of damage. In this chapter, we’ll begin to think formally about **CAUSAL INFERENCE** and introduce graphical causal models as a way to design and interpret regression models. The next chapter continues on this theme, describing some serious and common dangers of adding predictor variables, ending with a unifying framework for understanding the examples in both this chapter and the next.

Rethinking: Causal inference. Despite its central importance, there is no unified approach to causal inference yet in the sciences. There are even people who argue that cause does not really exist; it’s just a psychological illusion.⁸¹ And in complex dynamical systems, everything seems to cause everything else. “Cause” loses intuitive value. About one thing, however, there is general agreement: Causal inference always depends upon unverifiable assumptions. Another way to say this is that it’s always possible to imagine some way in which your inference about cause is mistaken, no matter how careful the design or analysis. A lot can be accomplished, despite this barrier.⁸²

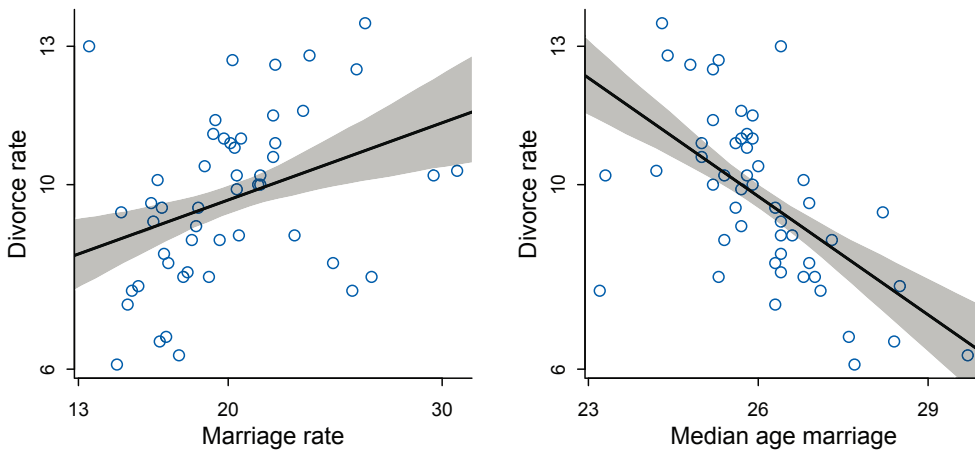


FIGURE 5.2. Divorce rate is associated with both marriage rate (left) and median age at marriage (right). Both predictor variables are standardized in this example. The average marriage rate across States is 20 per 1000 adults, and the average median age at marriage is 26 years.

5.1. Spurious association

Let's leave waffles behind, at least for the moment. An example that is easier to understand is the correlation between divorce rate and marriage rate (FIGURE 5.2). The rate at which adults marry is a great predictor of divorce rate, as seen in the left-hand plot in the figure. But does marriage *cause* divorce? In a trivial sense it obviously does: One cannot get a divorce without first getting married. But there's no reason high marriage rate must cause more divorce. It's easy to imagine high marriage rate indicating high cultural valuation of marriage and therefore being associated with *low* divorce rate.

Another predictor associated with divorce is the median age at marriage, displayed in the right-hand plot in FIGURE 5.2. Age at marriage is also a good predictor of divorce rate—higher age at marriage predicts less divorce. But there is no reason this has to be causal, either, unless age at marriage is very late and the spouses do not live long enough to get a divorce.

Let's load these data and standardize the variables of interest:

```
# load data and copy
library(rethinking)
data(WaffleDivorce)
d <- WaffleDivorce

# standardize variables
d$D <- standardize( d$Divorce )
d$M <- standardize( d$Marriage )
d$A <- standardize( d$MedianAgeMarriage )
```

R code
5.1

You can replicate the right-hand plot in the figure using a linear regression model:

$$\begin{aligned} D_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha + \beta_A A_i \\ \alpha &\sim \text{Normal}(0, 0.2) \\ \beta_A &\sim \text{Normal}(0, 0.5) \\ \sigma &\sim \text{Exponential}(1) \end{aligned}$$

D_i is the standardized (zero centered, standard deviation one) divorce rate for State i , and A_i is State i 's standardized median age at marriage. The linear model structure should be familiar from the previous chapter.

What about those priors? Since the outcome and the predictor are both standardized, the intercept α should end up very close to zero. What does the prior slope β_A imply? If $\beta_A = 1$, that would imply that a change of one standard deviation in age at marriage is associated likewise with a change of one standard deviation in divorce. To know whether or not that is a strong relationship, you need to know how big a standard deviation of age at marriage is:

```
R code 5.2 sd( d$MedianAgeMarriage )
```

```
[1] 1.24363
```

So when $\beta_A = 1$, a change of 1.2 years in median age at marriage is associated with a full standard deviation change in the outcome variable. That seems like an insanely strong relationship. The prior above thinks that only 5% of plausible slopes are more extreme than 1. We'll simulate from these priors in a moment, so you can see how they look in the outcome space.

To compute the approximate posterior, there are no new code tricks or techniques here. But I'll add comments to help explain the mass of code to follow.

```
R code 5.3 m5.1 <- quap(
  alist(
    D ~ dnorm( mu , sigma ) ,
    mu <- a + bA * A ,
    a ~ dnorm( 0 , 0.2 ) ,
    bA ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data = d )
```

To simulate from the priors, we can use `extract.prior` and `link` as in the previous chapter. I'll plot the lines over the range of 2 standard deviations for both the outcome and predictor. That'll cover most of the possible range of both variables.

```
R code 5.4 set.seed(10)
prior <- extract.prior( m5.1 )
mu <- link( m5.1 , post=prior , data=list( A=c(-2,2) ) )
plot( NULL , xlim=c(-2,2) , ylim=c(-2,2) )
for ( i in 1:50 ) lines( c(-2,2) , mu[i,] , col=col.alpha("black",0.4) )
```

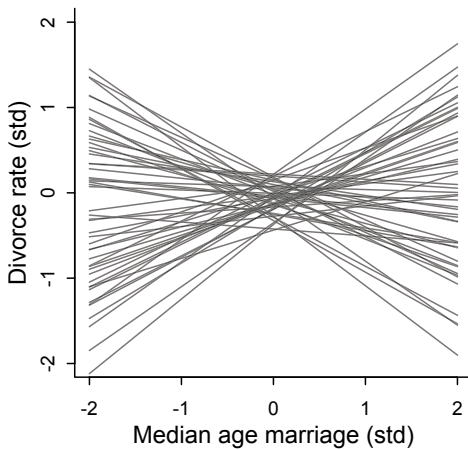


FIGURE 5.3. Plausible regression lines implied by the priors in `m5.1`. These are weakly informative priors in that they allow some implausibly strong relationships but generally bound the lines to possible ranges of the variables.

FIGURE 5.3 displays the result. You may wish to try some vaguer, flatter priors and see how quickly the prior regression lines become ridiculous.

Now for the posterior predictions. The procedure is exactly like the examples from the previous chapter: `link`, then summarize with `mean` and `PI`, and then `plot`.

```
# compute percentile interval of mean
A_seq <- seq( from=-3 , to=3.2 , length.out=30 )
mu <- link( m5.1 , data=list(A=A_seq) )
mu.mean <- apply( mu , 2, mean )
mu.PI <- apply( mu , 2 , PI )

# plot it all
plot( D ~ A , data=d , col=range(2) )
lines( A_seq , mu.mean , lwd=2 )
shade( mu.PI , A_seq )
```

R code
5.5

If you inspect the `precis` output, you'll see that posterior for β_A is reliably negative, as seen in FIGURE 5.2.

You can fit a similar regression for the relationship in the left-hand plot:

```
m5.2 <- quap(
  alist(
    D ~ dnorm( mu , sigma ) ,
    mu <- a + bM * M ,
    a ~ dnorm( 0 , 0.2 ) ,
    bM ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data = d )
```

R code
5.6

As you can see in the figure, this relationship isn't as strong as the previous one.

But merely comparing parameter means between different bivariate regressions is no way to decide which predictor is better. Both of these predictors could provide independent value, or they could be redundant, or one could eliminate the value of the other.

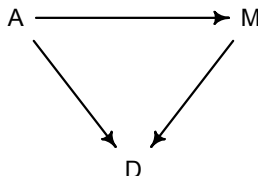
To make sense of this, we're going to have to think causally. And then, only after we've done some thinking, a bigger regression model that includes both age at marriage and marriage rate will help us.

5.1.1. Think before you regress. There are three observed variables in play: divorce rate (D), marriage rate (M), and the median age at marriage (A) in each State. The pattern we see in the previous two models and illustrated in [FIGURE 5.2](#) is symptomatic of a situation in which only one of the predictor variables, A in this case, has a *causal* impact on the outcome, D , even though both predictor variables are strongly associated with the outcome.

To understand this better, it is helpful to introduce a particular type of causal graph known as a **DAG**, short for **DIRECTED ACYCLIC GRAPH**. *Graph* means it is nodes and connections. *Directed* means the connections have arrows that indicate directions of causal influence. And *acyclic* means that causes do not eventually flow back on themselves. A DAG is a way of describing qualitative causal relationships among variables. It isn't as detailed as a full model description, but it contains information that a purely statistical model does not. Unlike a statistical model, a DAG will tell you the consequences of intervening to change a variable. But only if the DAG is correct. There is no inference without assumption.

The full framework for using DAGs to design and critique statistical models is complicated. So instead of smothering you in the whole framework right now, I'll build it up one example at a time. By the end of the next chapter, you'll have a set of simple rules that let you accomplish quite a lot of criticism. And then other applications will be introduced in later chapters.

Let's start with the basics. Here is a possible DAG for our divorce rate example:



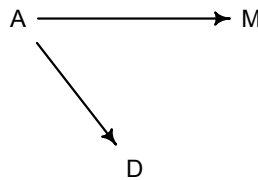
If you want to see the code to draw this, see the Overthinking box at the end of this section. It may not look like much, but this type of diagram does a lot of work. It represents a heuristic causal model. Like other models, it is an analytical assumption. The symbols A , M , and D are our observed variables. The arrows show directions of influence. What this DAG says is:

- (1) A directly influences D
- (2) M directly influences D
- (3) A directly influences M

These statements can then have further implications. In this case, age of marriage influences divorce in two ways. First it has a direct effect, $A \rightarrow D$. Perhaps a direct effect would arise because younger people change faster than older people and are therefore more likely to grow incompatible with a partner. Second, it has an indirect effect by influencing the marriage rate, which then influences divorce, $A \rightarrow M \rightarrow D$. If people get married earlier, then the marriage rate may rise, because there are more young people. Consider for example if an evil dictator forced everyone to marry at age 65. Since a smaller fraction of the population lives to 65 than to 25, forcing delayed marriage will also reduce the marriage rate. If marriage rate itself has any direct effect on divorce, maybe by making marriage more or less normative, then some of that direct effect could be the indirect effect of age at marriage.

To infer the strength of these different arrows, we need more than one statistical model. Model `m5.1`, the regression of D on A , tells us only that the *total* influence of age at marriage is strongly negative with divorce rate. The “total” here means we have to account for every path from A to D . There are two such paths in this graph: $A \rightarrow D$, a direct path, and $A \rightarrow M \rightarrow D$, an indirect path. In general, it is possible that a variable like A has no direct effect at all on an outcome like D . It could still be associated with D entirely through the indirect path. That type of relationship is known as **MEDIATION**, and we’ll have another example later.

As you’ll see however, the indirect path does almost no work in this case. How can we show that? We know from `m5.2` that marriage rate is positively associated with divorce rate. But that isn’t enough to tell us that the path $M \rightarrow D$ is positive. It could be that the association between M and D arises entirely from A ’s influence on both M and D . Like this:



This DAG is also consistent with the posterior distributions of models `m5.1` and `m5.2`. Why? Because both M and D “listen” to A . They have information from A . So when you inspect the association between D and M , you pick up that common information that they both got from listening to A . You’ll see a more formal way to deduce this, in the next chapter.

So which is it? Is there a direct effect of marriage rate, or rather is age at marriage just driving both, creating a spurious correlation between marriage rate and divorce rate? To find out, we need to consider carefully what each DAG implies. That’s what’s next.

Rethinking: What’s a cause? Questions of causation can become bogged down in philosophical debates. These debates are worth having. But they don’t usually intersect with statistical concerns. Knowing a cause in statistics means being able to correctly predict the consequences of an intervention. There are contexts in which even this is complicated. For example, it isn’t possible to directly change someone’s body weight. Changing someone’s body weight would mean intervening on another variable, like diet, and that variable would have other causal effects in addition. But being underweight can still be a legitimate cause of disease, even when we can’t intervene on it directly.

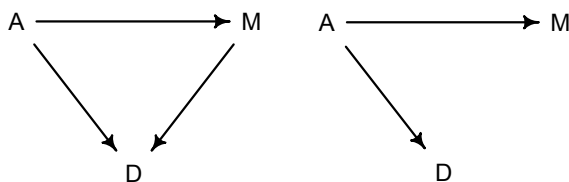
Overthinking: Drawing a DAG. There are several packages for drawing and analyzing DAGs. In this book, we’ll use `dagitty`. It is both an R package and something you can use in your internet browser: <http://www.dagitty.net/>. To draw the simple DAG you saw earlier in this section:

```
library(dagitty)
dag5.1 <- dagitty( "dag{ A -> D; A -> M; M -> D }" )
coordinates(dag5.1) <- list( x=c(A=0,D=1,M=2) , y=c(A=0,D=1,M=0) )
drawdag( dag5.1 )
```

R code
5.7

The `->` arrows in the DAG definition indicate directions of influence. The `coordinates` function lets you arrange the plot as you like.

5.1.2. Testable implications. How do we use data to compare multiple, plausible causal models? The first thing to consider is the **TESTABLE IMPLICATIONS** of each model. Consider the two DAGs we have so far considered:



Any DAG may imply that some variables are independent of others under certain conditions. These are the model’s testable implications, its **CONDITIONAL INDEPENDENCIES**. Conditional independencies come in two forms. First, they are statements of which variables should be associated with one another (or not) in the data. Second, they are statements of which variables become dis-associated when we *condition* on some other set of variables.

What does “conditioning” mean? Informally, conditioning on a variable Z means learning its value and then asking if X adds any additional information about Y . If learning X doesn’t give you any more information about Y , then we might say that Y is independent of X conditional on Z . This conditioning statement is sometimes written as: $Y \perp\!\!\!\perp X|Z$. This is very weird notation and any feelings of annoyance on your part are justified. We’ll work with this concept a lot, so don’t worry if it doesn’t entirely make sense right now. You’ll see examples very soon.

Let’s consider conditional independence in the context of the divorce example. What are the conditional independencies of the DAGs at the top of the page? How do we derive these conditional independencies? Finding conditional independencies is not hard, but also not at all obvious. With a little practice, it becomes very easy. The more general rules can wait until the next chapter. For now, let’s consider each DAG in turn and inspect the possibilities.

For the DAG on the left above, the one with three arrows, first note that every pair of variables is correlated. This is because there is a causal arrow between every pair. These arrows create correlations. So before we condition on anything, everything is associated with everything else. This is already a testable implication. We could write it:

$$D \not\perp\!\!\!\perp A \quad D \not\perp\!\!\!\perp M \quad A \not\perp\!\!\!\perp M$$

That $\not\perp\!\!\!\perp$ thing means “not independent of.” If we now look in the data and find that any pair of variables are not associated, then something is wrong with the DAG (assuming the data are correct). In these data, all three pairs are in fact strongly associated. Check for yourself. You can use `cor` to measure simple correlations. Correlations are sometimes terrible measures of association—many different patterns of association with different implications can produce the same correlation. But they do honest work in this case.

Are there any other testable implications for the first DAG above? No. It will be easier to see why, if we slide over to consider the second DAG, the one in which M has no influence on D . In this DAG, it is still true that all three variables are associated with one another. A is associated with D and M because it influences them both. And D and M are associated with one another, because M influences them both. They share a cause, and this leads them to be correlated with one another through that cause. But suppose we condition on A . All of the information in M that is relevant to predicting D is in A . So once we’ve conditioned on A , M tells us nothing more about D . So in the second DAG, a testable implication is that D is independent of M , conditional on A . In other words, $D \perp\!\!\!\perp M|A$. The same thing does not

happen with the first DAG. Conditioning on A does not make D independent of M , because M really influences D all by itself in this model.

In the next chapter, I'll show you the general rules for deducing these implications. For now, the `dagitty` package has the rules built in and can find the implications for you. Here's the code to define the second DAG and display the implied conditional independencies.

```
DMA_dag2 <- dagitty('dag{ D <- A -> M }')
impliedConditionalIndependencies( DMA_dag2 )
```

R code
5.8

$D \perp\!\!\!\perp M \mid A$

The first DAG has no conditional independencies. You can define it and check with this:

```
DMA_dag1 <- dagitty('dag{ D <- A -> M -> D }')
impliedConditionalIndependencies( DMA_dag1 )
```

R code
5.9

There are no conditional independencies, so there is no output to display.

Let's try to summarize. The testable implications of the first DAG are that all pairs of variables should be associated, whatever we condition on. The testable implications of the second DAG are that all pairs of variables should be associated, before conditioning on anything, but that D and M should be independent after conditioning on A . So the only implication that differs between these DAGs is the last one: $D \perp\!\!\!\perp M \mid A$.

To test this implication, we need a statistical model that conditions on A , so we can see whether that renders D independent of M . And that is what multiple regression helps with. It can address a useful *descriptive* question:

Is there any additional value in knowing a variable, once I already know all of the other predictor variables?

So for example once you fit a multiple regression to predict divorce using both marriage rate and age at marriage, the model addresses the questions:

- (1) After I already know marriage rate, what additional value is there in also knowing age at marriage?
- (2) After I already know age at marriage, what additional value is there in also knowing marriage rate?

The parameter estimates corresponding to each predictor are the (often opaque) answers to these questions. The questions above are descriptive, and the answers are also descriptive. It is only the derivation of the testable implications above that gives these descriptive results a causal meaning. But that meaning is still dependent upon believing the DAG.

Rethinking: “Control” is out of control. Very often, the question just above is spoken of as “statistical control,” as in *controlling for* the effect of one variable while estimating the effect of another. But this is sloppy language, as it implies too much. Statistical control is quite different from experimental control, as we'll explore more in the next chapter. The point here isn't to police language. Instead, the point is to observe the distinction between small world and large world interpretations. Since most people who use statistics are not statisticians, sloppy language like “control” can promote a sloppy culture of interpretation. Such cultures tend to overestimate the power of statistical methods, so resisting them can be difficult. Disciplining your own language may be enough. Disciplining another's language is hard to do, without seeming like a fastidious scold, as this very box must seem.

5.1.3. Multiple regression notation. Multiple regression formulas look a lot like the polynomial models at the end of the previous chapter—they add more parameters and variables to the definition of μ_i . The strategy is straightforward:

- (1) Nominate the predictor variables you want in the linear model of the mean.
- (2) For each predictor, make a parameter that will measure its conditional association with the outcome.
- (3) Multiply the parameter by the variable and add that term to the linear model.

Examples are always necessary, so here is the model that predicts divorce rate, using both marriage rate and age at marriage.

$$\begin{array}{ll}
 D_i \sim \text{Normal}(\mu_i, \sigma) & \text{[probability of data]} \\
 \mu_i = \alpha + \beta_M M_i + \beta_A A_i & \text{[linear model]} \\
 \alpha \sim \text{Normal}(0, 0.2) & \text{[prior for } \alpha\text{]} \\
 \beta_M \sim \text{Normal}(0, 0.5) & \text{[prior for } \beta_M\text{]} \\
 \beta_A \sim \text{Normal}(0, 0.5) & \text{[prior for } \beta_A\text{]} \\
 \sigma \sim \text{Exponential}(1) & \text{[prior for } \sigma\text{]}
 \end{array}$$

You can use whatever symbols you like for the parameters and variables, but here I've chosen R for marriage rate and A for age at marriage, reusing these symbols as subscripts for the corresponding parameters. But feel free to use whichever symbols reduce the load on your own memory.

So what does it mean to assume $\mu_i = \alpha + \beta_M M_i + \beta_A A_i$? Mechanically, it means that the expected outcome for any State with marriage rate M_i and median age at marriage A_i is the sum of three independent terms. If you are like most people, this is still pretty mysterious. The mechanical meaning of the equation doesn't map onto a unique causal meaning. Let's take care of the mechanical bits first, before returning to interpretation.

Overthinking: Compact notation and the design matrix. Often, linear models are written using a compact form like:

$$\mu_i = \alpha + \sum_{j=1}^n \beta_j x_{ji}$$

where j is an index over predictor variables and n is the number of predictor variables. This may be read as *the mean is modeled as the sum of an intercept and an additive combination of the products of parameters and predictors*. Even more compactly, using matrix notation:

$$\mathbf{m} = \mathbf{X}\mathbf{b}$$

where $\mathbf{m}$ is a vector of predicted means, one for each row in the data, $\mathbf{b}$ is a (column) vector of parameters, one for each predictor variable, and $\mathbf{X}$ is a matrix. This matrix is called a *design matrix*. It has as many rows as the data, and as many columns as there are predictors plus one. So $\mathbf{X}$ is basically a data frame, but with an extra first column. The extra column is filled with 1s. These 1s are multiplied by the first parameter, which is the intercept, and so return the unmodified intercept. When $\mathbf{X}$ is matrix-multiplied by $\mathbf{b}$, you get the predicted means. In R notation, this operation is `X %*% b`.

We're not going to use the design matrix approach. But it's good to recognize it, and sometimes it can save you a lot of work. For example, for linear regressions, there is a nice matrix formula for the maximum likelihood (or *least squares*) estimates. Most statistical software exploits that formula.

5.1.4. Approximating the posterior. To fit this model to the divorce data, we just expand the linear model. Here's the model definition again, with the code on the right-hand side:

$D_i \sim \text{Normal}(\mu_i, \sigma)$	<code>D ~ dnorm(mu, sigma)</code>
$\mu_i = \alpha + \beta_M M_i + \beta_A A_i$	<code>mu <- a + bM*M + bA*A</code>
$\alpha \sim \text{Normal}(0, 0.2)$	<code>a ~ dnorm(0, 0.2)</code>
$\beta_M \sim \text{Normal}(0, 0.5)$	<code>bM ~ dnorm(0, 0.5)</code>
$\beta_A \sim \text{Normal}(0, 0.5)$	<code>bA ~ dnorm(0, 0.5)</code>
$\sigma \sim \text{Exponential}(1)$	<code>sigma ~ dexp(1)</code>

And here is the quap code to approximate the posterior distribution:

```
m5.3 <- quap(
  alist(
    D ~ dnorm( mu , sigma ) ,
    mu <- a + bM*M + bA*A ,
    a ~ dnorm( 0 , 0.2 ) ,
    bM ~ dnorm( 0 , 0.5 ) ,
    bA ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data = d )
precis( m5.3 )
```

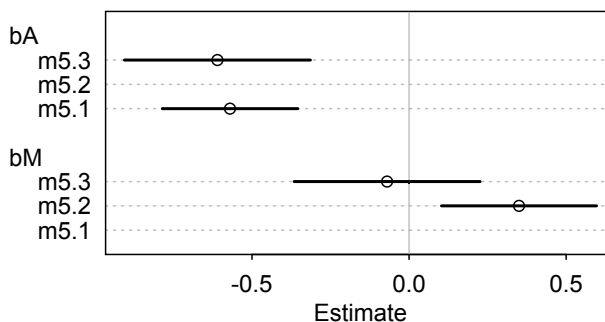
R code
5.10

	mean	sd	5.5%	94.5%
a	0.00	0.10	-0.16	0.16
bM	-0.07	0.15	-0.31	0.18
bA	-0.61	0.15	-0.85	-0.37
sigma	0.79	0.08	0.66	0.91

The posterior mean for marriage rate, b_M , is now close to zero, with plenty of probability of both sides of zero. The posterior mean for age at marriage, b_A , is essentially unchanged. It will help to visualize the posterior distributions for all three models, focusing just on the slope parameters β_A and β_M :

```
plot( coeftab(m5.1,m5.2,m5.3), par=c("bA","bM") )
```

R code
5.11



The posterior means are shown by the points and the 89% compatibility intervals by the solid horizontal lines. Notice how b_A doesn't move, only grows a bit more uncertain, while

bM is only associated with divorce when age at marriage is missing from the model. You can interpret these distributions as saying:

Once we know median age at marriage for a State, there is little or no additional predictive power in also knowing the rate of marriage in that State.

In that weird notation, $D \perp\!\!\!\perp M|A$. This tests the implication of the second DAG from earlier. Since the first DAG did not imply this result, it is out.

Note that this does not mean that there is no value in knowing marriage rate. Consistent with the earlier DAG, if you didn't have access to age-at-marriage data, then you'd definitely find value in knowing the marriage rate. M is predictive but not causal. Assuming there are no other causal variables missing from the model (more on that in the next chapter), this implies there is no important direct causal path from marriage rate to divorce rate. The association between marriage rate and divorce rate is spurious, caused by the influence of age of marriage on both marriage rate and divorce rate. I'll leave it to the reader to investigate the relationship between age at marriage, A , and marriage rate, M , to complete the picture.

But how did model m5.3 achieve the inference that marriage rate adds no additional information, once we know age at marriage? Let's draw some pictures.

Overthinking: Simulating the divorce example. The divorce data are real data. See the sources in `?WaffleDivorce`. But it is useful to simulate the kind of causal relationships shown in the previous DAG: $M \leftarrow A \rightarrow D$. Every DAG implies a simulation, and such simulations can help us design models to correctly infer relationships among variables. In this case, you just need to simulate each of the three variables:

R code
5.12

```
N <- 50 # number of simulated States
age <- rnorm( N )           # sim A
mar <- rnorm( N , -age )    # sim A -> M
div <- rnorm( N , age )     # sim A -> D
```

Now if you use these variables in models m5.1, m5.2, and m5.3, you'll see the same pattern of posterior inferences. It is also possible to simulate that both A and M influence D : `div <- rnorm(N, age + mar)`. In that case, a naive regression of D on A will overestimate the influence of A , just like a naive regression of D on M will overestimate the importance of M . The multiple regression will help sort things out for you in this situation as well. But interpreting the parameter estimates will always depend upon what you believe about the causal model, because typically several (or very many) causal models are consistent with any one set of parameter estimates. We'll discuss this later in the chapter as [MARKOV EQUIVALENCE](#).

5.1.5. Plotting multivariate posteriors. Let's pause for a moment, before moving on. There are a lot of moving parts here: three variables, some strange DAGs, and three models. If you feel at all confused, it is only because you are paying attention.

It will help to visualize the model's inferences. Visualizing the posterior distribution in simple bivariate regressions, like those in the previous chapter, is easy. There's only one predictor variable, so a single scatterplot can convey a lot of information. And so in the previous chapter we used scatters of the data. Then we overlaid regression lines and intervals to both (1) visualize the size of the association between the predictor and outcome and (2) to get a crude sense of the ability of the model to predict the individual observations.

With multivariate regression, you'll need more plots. There is a huge literature detailing a variety of plotting techniques that all attempt to help one understand multiple linear

regression. None of these techniques is suitable for all jobs, and most do not generalize beyond linear regression. So the approach I take here is to instead help you compute whatever you need from the model. I offer three examples of interpretive plots:

- (1) *Predictor residual plots*. These plots show the outcome against *residual* predictor values. They are useful for understanding the statistical model, but not much else.
- (2) *Posterior prediction plots*. These show model-based predictions against raw data, or otherwise display the error in prediction. They are tools for checking fit and assessing predictions. They are not causal tools.
- (3) *Counterfactual plots*. These show the implied predictions for imaginary experiments. These plots allow you to explore the causal implications of manipulating one or more variables.

Each of these plot types has its advantages and deficiencies, depending upon the context and the question of interest. In the rest of this section, I show you how to manufacture each of these in the context of the divorce data.

5.1.5.1. *Predictor residual plots*. A predictor residual is the average prediction error when we use all of the other predictor variables to model a predictor of interest. That's a complicated concept, so we'll go straight to the example, where it will make sense. The benefit of computing these things is that, once plotted against the outcome, we have a bivariate regression that has already conditioned on all of the other predictor variables. It leaves the variation that is not expected by the model of the mean, μ , as a function of the other predictors.

In our model of divorce rate, we have two predictors: (1) marriage rate M and (2) median age at marriage A . To compute predictor residuals for either, we just use the other predictor to model it. So for marriage rate, this is the model we need:

$$\begin{aligned} M_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha + \beta A_i \\ \alpha &\sim \text{Normal}(0, 0.2) \\ \beta &\sim \text{Normal}(0, 0.5) \\ \sigma &\sim \text{Exponential}(1) \end{aligned}$$

As before, M is marriage rate and A is median age at marriage. Note that since we standardized both variables, we already expect the mean α to be around zero, as before. So I'm reusing the same priors as earlier. This code will approximate the posterior:

```
m5.4 <- quap(
  alist(
    M ~ dnorm( mu , sigma ) ,
    mu <- a + bAM * A ,
    a ~ dnorm( 0 , 0.2 ) ,
    bAM ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data = d )
```

R code
5.13

And then we compute the **RESIDUALS** by subtracting the observed marriage rate in each State from the predicted rate, based upon the model above:

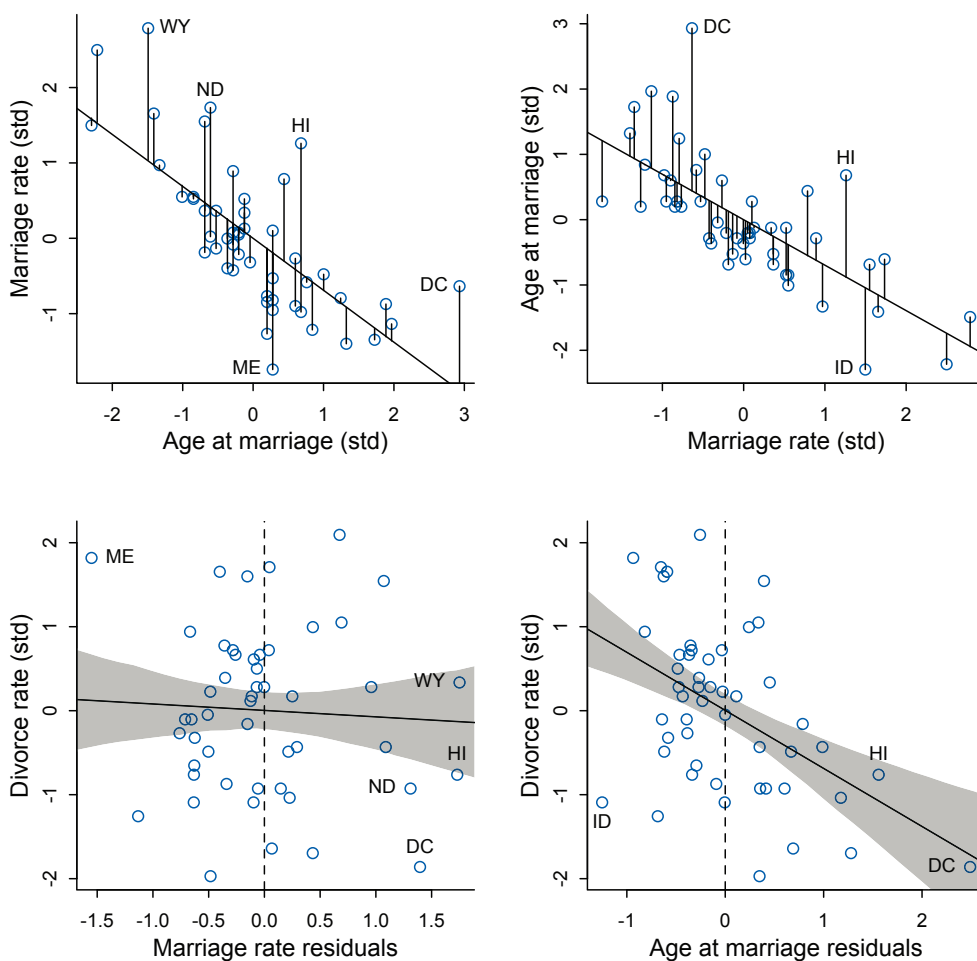


FIGURE 5.4. Understanding multiple regression through residuals. The top row shows each predictor regressed on the other predictor. The lengths of the line segments connecting the model's expected value of the outcome, the regression line, and the actual value are the *residuals*. In the bottom row, divorce rate is regressed on the residuals from the top row. Bottom left: Residual variation in marriage rate shows little association with divorce rate. Bottom right: Divorce rate on age at marriage residuals, showing remaining variation, and this variation is associated with divorce rate.

```
R code
5.14 mu <- link(m5.4)
      mu_mean <- apply( mu , 2 , mean )
      mu_resid <- d$M - mu_mean
```

When a residual is positive, that means that the observed rate was in excess of what the model expects, given the median age at marriage in that State. When a residual is negative, that means the observed rate was below what the model expects. In simpler terms, States with

positive residuals have high marriage rates for their median age of marriage, while States with negative residuals have low rates for their median age of marriage. It'll help to plot the relationship between these two variables, and show the residuals as well. In [FIGURE 5.4](#), upper left, I show $m5.4$ along with line segments for each residual. Notice that the residuals are variation in marriage rate that is left over, after taking out the purely linear relationship between the two variables.

Now to use these residuals, let's put them on a horizontal axis and plot them against the actual outcome of interest, divorce rate. In [FIGURE 5.4](#) also (lower left), I plot these residuals against divorce rate, overlaying the linear regression of the two variables. You can think of this plot as displaying the linear relationship between divorce and marriage rates, having conditioned already on median age of marriage. The vertical dashed line indicates marriage rate that exactly matches the expectation from median age at marriage. So States to the right of the line have higher marriage rates than expected. States to the left of the line have lower rates. Average divorce rate on both sides of the line is about the same, and so the regression line demonstrates little relationship between divorce and marriage rates.

The same procedure works for the other predictor. The top right plot in [FIGURE 5.4](#) shows the regression of A on M and the residuals. In the lower right, these residuals are used to predict divorce rate. States to the right of the vertical dashed line have older-than-expected median age at marriage, while those to the left have younger-than-expected median age at marriage. Now we find that the average divorce rate on the right is lower than the rate on the left, as indicated by the regression line. States in which people marry older than expected for a given rate of marriage tend to have less divorce.

So what's the point of all of this? There's conceptual value in seeing the model-based predictions displayed against the outcome, after subtracting out the influence of other predictors. The plots in [FIGURE 5.4](#) do this. But this procedure also brings home the message that regression models measure the remaining association of each predictor with the outcome, after already knowing the other predictors. In computing the predictor residual plots, you had to perform those calculations yourself. In the unified multivariate model, it all happens automatically. Nevertheless, it is useful to keep this fact in mind, because regressions can behave in surprising ways as a result. We'll have an example soon.

Linear regression models do all of this simultaneous measurement with a very specific additive model of how the variables relate to one another. But predictor variables can be related to one another in non-additive ways. The basic logic of statistical conditioning does not change in those cases, but the details definitely do, and these residual plots cease to be useful. Luckily there are other ways to understand a model. That's where we turn next.

Rethinking: Residuals are parameters, not data. There is a tradition, especially in parts of biology, of using residuals from one model as data in another model. For example, a biologist might regress brain size on body size and then use the brain size residuals as data in another model. This procedure is always a mistake. Residuals are not known. They are parameters, variables with unobserved values. Treating them as known values throws away uncertainty. The right way to adjust for body size is to include it in the same model,⁸³ preferably a model designed in light of an explicit causal model.

5.1.5.2. Posterior prediction plots. It's important to check the model's implied predictions against the observed data. This is what you did in [Chapter 3](#), when you simulated globe tosses, averaging over the posterior, and comparing the simulated results to the observed. These kinds of checks are useful in many ways. For now, we'll focus on two uses.

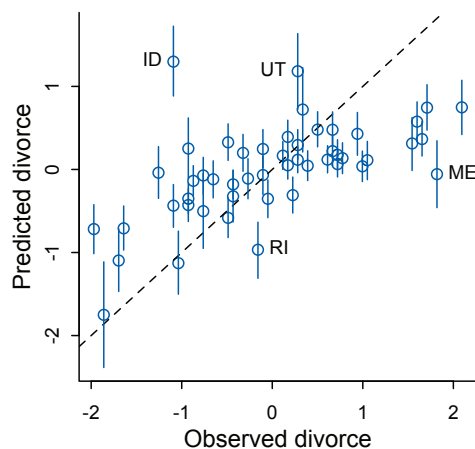


FIGURE 5.5. Posterior predictive plot for the multivariate divorce model, `m5.3`. The horizontal axis is the observed divorce rate in each State. The vertical axis is the model's posterior predicted divorce rate, given each State's median age at marriage and marriage rate. The blue line segments are 89% compatibility intervals. The diagonal line shows where posterior predictions exactly match the sample.

- (1) Did the model correctly approximate the posterior distribution? Golems do make mistakes, as do golem engineers. Errors can be more easily diagnosed by comparing implied predictions to the raw data. Some caution is required, because not all models try to exactly match the sample. But even then, you'll know what to expect from a successful approximation. You'll see some examples later ([Chapter 13](#)).
- (2) How does the model fail? Models are useful fictions. So they always fail in some way. Sometimes, a model fits correctly but is still so poor for our purposes that it must be discarded. More often, a model predicts well in some respects, but not in others. By inspecting the individual cases where the model makes poor predictions, you might get an idea of how to improve it. The difficulty is that this process is essentially creative and relies upon the analyst's domain expertise. No robot can (yet) do it for you. It also risks chasing noise, a topic we'll focus on in later chapters.

How could we produce a simple posterior predictive check in the divorce example? Let's begin by simulating predictions, averaging over the posterior.

```
R code
5.15 # call link without specifying new data
      # so it uses original data
      mu <- link( m5.3 )

      # summarize samples across cases
      mu_mean <- apply( mu , 2 , mean )
      mu_PI <- apply( mu , 2 , PI )

      # simulate observations
      # again no new data, so uses original data
      D_sim <- sim( m5.3 , n=1e4 )
      D_PI <- apply( D_sim , 2 , PI )
```

This code is similar to what you've seen before, but now using the original observed data.

For multivariate models, there are many different ways to display these simulations. The simplest is to just plot predictions against observed. This code will do that, and then add a line to show perfect prediction and line segments for the confidence interval of each prediction:


```
plot( mu_mean ~ d$D , col=rangi2 , ylim=range(mu_PI) ,
      xlab="Observed divorce" , ylab="Predicted divorce" )
abline( a=0 , b=1 , lty=2 )
for ( i in 1:nrow(d) ) lines( rep(d$D[i],2) , mu_PI[,i] , col=rangi2 )
```

R code
5.16

The resulting plot appears in [FIGURE 5.5](#). It's easy to see from this arrangement of the simulations that the model under-predicts for States with very high divorce rates while it over-predicts for States with very low divorce rates. That's normal. This is what regression does—it is skeptical of extreme values, so it expects regression towards the mean. But beyond this general regression to the mean, some States are very frustrating to the model, lying very far from the diagonal. I've labeled some points like this, including Idaho (ID) and Utah (UT), both of which have much lower divorce rates than the model expects them to have. The easiest way to label a few select points is to use `identify`:

```
identify( x=d$D , y=mu_mean , labels=d$Loc )
```

R code
5.17

After executing the line of code above, R will wait for you to click near a point in the active plot window. It'll then place a label near that point, on the side you choose. When you are done labeling points, press your right mouse button (or press `ESC`, on some platforms).

What is unusual about Idaho and Utah? Both of these States have large proportions of members of the Church of Jesus Christ of Latter-day Saints. Members of this church have low rates of divorce, wherever they live. This suggests that having a finer view on the demographic composition of each State, beyond just median age at marriage, would help.

Rethinking: Stats, huh, yeah what is it good for? Often people want statistical modeling to do things that statistical modeling cannot do. For example, we'd like to know whether an effect is “real” or rather spurious. Unfortunately, modeling merely quantifies uncertainty in the precise way that the model understands the problem. Usually answers to large world questions about truth and causation depend upon information not included in the model. For example, any observed correlation between an outcome and predictor could be eliminated or reversed once another predictor is added to the model. But if we cannot think of the right variable, we might never notice. Therefore all statistical models are vulnerable to and demand critique, regardless of the precision of their estimates and apparent accuracy of their predictions. Rounds of model criticism and revision embody the real tests of scientific hypotheses. A true hypothesis will pass and fail many statistical “tests” on its way to acceptance.

Overthinking: Simulating spurious association. One way that spurious associations between a predictor and outcome can arise is when a truly causal predictor, call it x_{real} , influences both the outcome, y , and a spurious predictor, x_{spur} . This can be confusing, however, so it may help to simulate this scenario and see both how the spurious data arise and prove to yourself that multiple regression can reliably indicate the right predictor, x_{real} . So here's a very basic simulation:

```
N <- 100                                # number of cases
x_real <- rnorm( N )                     # x_real as Gaussian with mean 0 and stddev 1
x_spur <- rnorm( N , x_real )           # x_spur as Gaussian with mean=x_real
y <- rnorm( N , x_real )                 # y as Gaussian with mean=x_real
d <- data.frame(y,x_real,x_spur)        # bind all together in data frame
```

R code
5.18

Now the data frame `d` has 100 simulated cases. Because `x_real` influences both `y` and `x_spur`, you can think of `x_spur` as another outcome of `x_real`, but one which we mistake as a potential predictor of `y`. As a result, both `x_real` and `x_spur` are correlated with `y`. You can see this in the scatterplots from `pairs(d)`. But when you include both `x` variables in a linear regression predicting `y`, the posterior mean for the association between `y` and `x_spur` will be close to zero.

5.1.5.3. Counterfactual plots. A second sort of inferential plot displays the causal implications of the model. I call these plots **COUNTERFACTUAL**, because they can be produced for any values of the predictor variables you like, even unobserved combinations like very high median age of marriage and very high marriage rate. There are no States with this combination, but in a counterfactual plot, you can ask the model for a prediction for such a State, asking questions like “What would Utah’s divorce rate be, if it’s median age at marriage were higher?” Used with clarity of purpose, counterfactual plots help you understand the model, as well as generate predictions for imaginary interventions and compute how much some observed outcome could be attributed to some cause.

Note that the term “counterfactual” is highly overloaded in statistics and philosophy. It hardly ever means the same thing when used by different authors. Here, I use it to indicate some computation that makes use of the structural causal model, going beyond the posterior distribution. But it could refer to questions about both the past and the future.

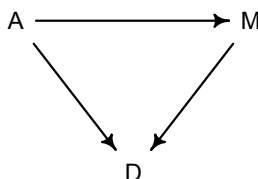
The simplest use of a counterfactual plot is to see how the outcome would change as you change one predictor at a time. If some predictor `X` took on a new value for one or more cases in our data, how would the outcome `Y` have changed? Changing just one predictor `X` might also change other predictors, depending upon the causal model. Suppose for example that you pay young couples to postpone marriage until they are 35 years old. Surely this will also decrease the number of couples who ever get married—some people will die before turning 35, among other reasons—decreasing the overall marriage rate. An extraordinary and evil degree of control over people would be necessary to really hold marriage rate constant while forcing everyone to marry at a later age.

So let’s see how to generate plots of model predictions that take the causal structure into account. The basic recipe is:

- (1) Pick a variable to manipulate, the intervention variable.
- (2) Define the range of values to set the intervention variable to.
- (3) For each value of the intervention variable, and for each sample in posterior, use the causal model to simulate the values of other variables, including the outcome.

In the end, you end up with a posterior distribution of counterfactual outcomes that you can plot and summarize in various ways, depending upon your goal.

Let’s see how to do this for the divorce model. Again we take this DAG as given:



To simulate from this, we need more than the DAG. We also need a set of functions that tell us how each variable is generated. For simplicity, we’ll use Gaussian distributions for each variable, just like in model `m5.3`. But model `m5.3` ignored the assumption that `A` influences

M . We didn't need that to estimate $A \rightarrow D$. But we do need it to predict the consequences of manipulating A , because some of the effect of A acts through M .

To estimate the influence of A on M , all we need is to regress A on M . There are no other variables in the DAG creating an association between A and M . We can just add this regression to the quap model, running two regressions at the same time:

```
data(WaffleDivorce)
d <- list()
d$A <- standardize( WaffleDivorce$MedianAgeMarriage )
d$D <- standardize( WaffleDivorce$Divorce )
d$M <- standardize( WaffleDivorce$Marriage )

m5.3_A <- quap(
  alist(
    ## A -> D <- M
    D ~ dnorm( mu , sigma ) ,
    mu <- a + bM*M + bA*A ,
    a ~ dnorm( 0 , 0.2 ) ,
    bM ~ dnorm( 0 , 0.5 ) ,
    bA ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 ) ,
    ## A -> M
    M ~ dnorm( mu_M , sigma_M ) ,
    mu_M <- aM + bAM*A ,
    aM ~ dnorm( 0 , 0.2 ) ,
    bAM ~ dnorm( 0 , 0.5 ) ,
    sigma_M ~ dexp( 1 )
  ) , data = d )
```

R code
5.19

Look at the `precis(5.3_A)` summary. You'll see that M and A are strongly negatively associated. If we interpret this causally, it indicates that manipulating A reduces M .

The goal is to simulate what would happen, if we manipulate A . So next we define a range of values for A .

```
A_seq <- seq( from=-2 , to=2 , length.out=30 )
```

R code
5.20

This defines a list of 30 imaginary interventions, ranging from 2 standard deviations below and 2 above the mean. Now we can use `sim`, which you met in the previous chapter, to simulate observations from model `m5.3_A`. But this time we'll tell it to simulate both M and D , in that order. Why in that order? Because we have to simulate the influence of A on M before we simulate the joint influence of A and M on D . The `vars` argument to `sim` tells it both which observables to simulate and in which order.

```
# prep data
sim_dat <- data.frame( A=A_seq )

# simulate M and then D, using A_seq
s <- sim( m5.3_A , data=sim_dat , vars=c("M","D") )
```

R code
5.21

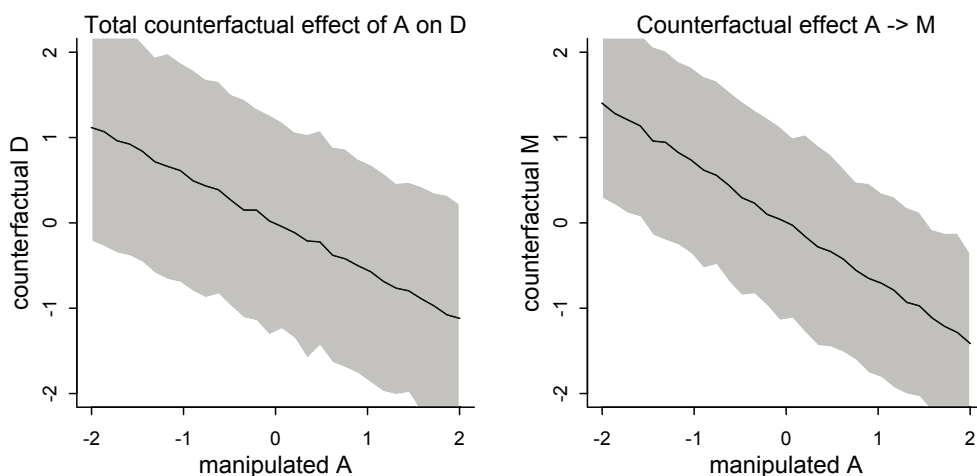


FIGURE 5.6. Counterfactual plots for the multivariate divorce model, m5.3. These plots visualize the predicted effect of manipulating age at marriage A on divorce rate D . Left: Total causal effect of manipulating A (horizontal) on D . This plot contains both paths, $A \rightarrow D$ and $A \rightarrow M \rightarrow D$. Right: Simulated values of M show the estimated influence $A \rightarrow M$.

That's all there is to it. But do at least glance at the Overthinking box at the end of this section, where I show you the individual steps, so you can perform this kind of counterfactual simulation for any model fit with any software. Now to plot the predictions:

```
R code
5.22 plot( sim_dat$A , colMeans(s$D) , ylim=c(-2,2) , type="l" ,
      xlab="manipulated A" , ylab="counterfactual D" )
      shade( apply(s$D,2,PI) , sim_dat$A )
      mtext( "Total counterfactual effect of A on D" )
```

The resulting plot is shown in [FIGURE 5.6](#) (left side). This predicted trend in D includes both paths: $A \rightarrow D$ and $A \rightarrow M \rightarrow D$. We found previously that $M \rightarrow D$ is very small, so the second path doesn't contribute much to the trend. But if M were to strongly influence D , the code above would include the effect. The counterfactual simulation also generated values for M . These are shown on the right in [FIGURE 5.6](#). The object `s` from the code above includes these simulated M values. Try to reproduce the figure yourself.

Of course these calculations also permit numerical summaries. For example, the expected causal effect of increasing median age at marriage from 20 to 30 is:

```
R code
5.23 # new data frame, standardized to mean 26.1 and std dev 1.24
sim2_dat <- data.frame( A=(c(20,30)-26.1)/1.24 )
s2 <- sim( m5.3_A , data=sim2_dat , vars=c("M","D") )
mean( s2$D[,2] - s2$D[,1] )
```

```
[1] -4.591425
```

This is a huge effect of four and one half standard deviations, probably impossibly large.

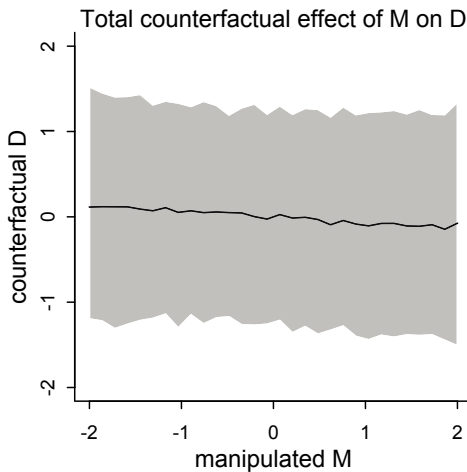
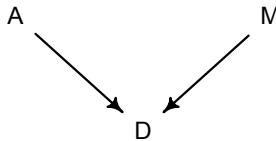


FIGURE 5.7. The counterfactual effect of manipulating marriage rate M on divorce rate D . Since $M \rightarrow D$ was estimated to be very small, there is no strong trend here. By manipulating M , we break the influence of A on M , and this removes the association between M and D .

The trick with simulating counterfactuals is to realize that when we manipulate some variable X , we break the causal influence of other variables on X . This is the same as saying we modify the DAG so that no arrows enter X . Suppose for example that we now simulate the effect of manipulating M . This implies the DAG:



The arrow $A \rightarrow M$ is deleted, because if we control the values of M , then A no longer influences it. It's like a perfectly controlled experiment. Now we can modify the code above to simulate the counterfactual result of manipulating M . We'll simulate a counterfactual for an average state, with $A = 0$, and see what changing M does.

```

sim_dat <- data.frame( M=seq(from=-2,to=2,length.out=30) , A=0 )
s <- sim( m5.3_A , data=sim_dat , vars="D" )

plot( sim_dat$M , colMeans(s) , ylim=c(-2,2) , type="l" ,
      xlab="manipulated M" , ylab="counterfactual D" )
shade( apply(s,2,PI) , sim_dat$M )
mtext( "Total counterfactual effect of M on D" )
  
```

R code
5.24

We only simulate D now—note the `vars` argument to `sim()` in the code above. We don't simulate A , because M doesn't influence it. I show this plot in [FIGURE 5.7](#). This trend is less strong, because there is no evidence for a strong influence of M on D .

In more complex models with many potential paths, the same strategy will compute counterfactuals for an exposure of interest. But as you'll see in later examples, often it is simply not possible to estimate a plausible, un-confounded causal effect of some exposure X on some outcome Y . But even in those cases, there are still important counterfactuals to consider. So we'll return to this theme in future chapters.

Overthinking: Simulating counterfactuals. The example in this section used `sim()` to hide the details. But simulating counterfactuals on your own is not hard. It just uses the model definition. Assume we've already fit model `m5.3_A`, the model that includes both causal paths $A \rightarrow D$ and $A \rightarrow M \rightarrow D$. We define a range of values that we want to assign to A :

```
R code 5.25
A_seq <- seq( from=-2 , to=2 , length.out=30 )
```

Next we need to extract the posterior samples, because we'll simulate observations for each set of samples. Then it really is just a matter of using the model definition with the samples, as in previous examples. The model defines the distribution of M . We just convert that definition to the corresponding simulation function, which is `rnorm` in this case:

```
R code 5.26
post <- extract.samples( m5.3_A )
M_sim <- with( post , sapply( 1:30 ,
  function(i) rnorm( 1e3 , aM + bAM*A_seq[i] , sigma_M ) ) )
```

I used the `with` function, which saves us having to type `post$` in front of every parameter name. The linear model inside `rnorm` comes right out of the model definition. This produces a matrix of values, with samples in rows and cases corresponding to the values in `A_seq` in the columns. Now that we have simulated values for M , we can simulate D too:

```
R code 5.27
D_sim <- with( post , sapply( 1:30 ,
  function(i) rnorm( 1e3 , a + bA*A_seq[i] + bM*M_sim[,i] , sigma ) ) )
```

If you plot `A_seq` against the column means of `D_sim`, you'll see the same result as before. In complex models, there might be many more variables to simulate. But the basic procedure is the same.

5.2. Masked relationship

The divorce rate example demonstrates that multiple predictor variables are useful for knocking out spurious association. A second reason to use more than one predictor variable is to measure the direct influences of multiple factors on an outcome, when none of those influences is apparent from bivariate relationships. This kind of problem tends to arise when there are two predictor variables that are correlated with one another. However, one of these is positively correlated with the outcome and the other is negatively correlated with it.

You'll consider this kind of problem in a new data context, information about the composition of milk across primate species, as well as some facts about those species, like body mass and brain size.⁸⁴ Milk is a huge investment, being much more expensive than gestation. Such an expensive resource is likely adjusted in subtle ways, depending upon the physiological and development details of each mammal species. Let's load the data into R first:

```
R code 5.28
library(rethinking)
data(milk)
d <- milk
str(d)
```

You should see in the structure of the data frame that you have 29 rows for 8 variables. The variables we'll consider for now are `kcal.per.g` (kilocalories of energy per gram of milk),

mass (average female body mass, in kilograms), and `neocortex.perc` (percent of total brain mass that is neocortex mass).

A popular hypothesis has it that primates with larger brains produce more energetic milk, so that brains can grow quickly. Answering questions of this sort consumes a lot of effort in evolutionary biology, because there are many subtle statistical issues that arise when comparing species. It doesn't help that many biologists have no reference model other than a series of regressions, and so the output of the regressions is not really interpretable. The causal meaning of statistical estimates always depends upon information outside the data.

We won't solve these problems here. But we will explore a useful example. The question here is to what extent energy content of milk, measured here by kilocalories, is related to the percent of the brain mass that is neocortex. Neocortex is the gray, outer part of the brain that is especially elaborate in some primates. We'll end up needing female body mass as well, to see the masking that hides the relationships among the variables. Let's standardize these three variables. As in previous examples, standardizing helps us both get a reliable approximation of the posterior as well as build reasonable priors.

```
d$K <- standardize( d$kcals.per.g )
d$N <- standardize( d$neocortex.perc )
d$M <- standardize( log(d$mass) )
```

R code
5.29

The first model to consider is the simple bivariate regression between kilocalories and neocortex percent. You already know how to set up this regression. In mathematical form:

$$K_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta_N N_i$$

where K is standardized kilocalories and N is standardized neocortex percent. We still need to consider the priors. But first let's just try to run this as a quap model with some vague priors, because there is another key modeling issue to address first.

```
m5.5_draft <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bN*N ,
    a ~ dnorm( 0 , 1 ) ,
    bN ~ dnorm( 0 , 1 ) ,
    sigma ~ dexp( 1 )
  ) , data=d )
```

R code
5.30

When you execute this code, you'll get a confusing error message:

```
Error in quap(alist(K ~ dnorm(mu, sigma), mu <- a + bN * N, a ~ dnorm(0, :
  initial value in 'vmmmin' is not finite
The start values for the parameters were invalid. This could be caused by
missing values (NA) in the data or by start values outside the parameter
constraints. If there are no NAs, try using explicit start values.
```

What has gone wrong here? This particular error message means that the model didn't return a valid probability for even the starting parameter values. In this case, the culprit is the missing values in the N variable. Take a look inside the original variable and see for yourself:

```
R code
5.31 d$neocortex.perc
```

```
[1] 55.16    NA     NA     NA     NA 64.54 64.54 67.64    NA 68.85 58.85 61.69
[13] 60.32    NA     NA 69.97    NA 70.41    NA 73.40    NA 67.53    NA 71.26
[25] 72.60    NA 70.24 76.30 75.49
```

Each NA in the output is a missing value. If you pass a vector like this to a likelihood function like `dnorm`, it doesn't know what to do. After all, what's the probability of a missing value? Whatever the answer, it isn't a number, and so `dnorm` returns a NaN. Unable to even get started, `quap` (or rather `optim`, which does the real work) gives up and barks about some weird thing called `vmmin` not being finite. This kind of opaque error message is unfortunately the norm in R. The additional part of the message suggesting NA values might be responsible is just `quap` taking a guess.

This is easy to fix. What you need to do here is manually drop all the cases with missing values. This is known as a **COMPLETE CASE ANALYSIS**. More automated model fitting commands, like `lm` and `glm`, will silently drop such cases for you. But this isn't always a good thing. First, its validity depends upon the process that caused these particular values to go missing. In [Chapter 15](#), you'll explore this in much more depth. Second, once you start comparing models, you must compare models fit to the same data. If some variables have missing values that others do not, automated tools will silently produce misleading comparisons.

Let's march forward for now, dropping any cases with missing values. It's worth learning how to do this yourself. To make a new data frame with only complete cases, use:

```
R code
5.32 dcc <- d[ complete.cases(d$K,d$N,d$M) , ]
```

This makes a new data frame, `dcc`, that consists of the 17 rows from `d` that have no missing values in any of the variables listed inside `complete.cases`. Now let's work with the new data frame. All that is new in the code is using `dcc` instead of `d`:

```
R code
5.33 m5.5_draft <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bN*N ,
    a ~ dnorm( 0 , 1 ) ,
    bN ~ dnorm( 0 , 1 ) ,
    sigma ~ dexp( 1 )
  ) , data=dcc )
```

Before considering the posterior predictions, let's consider those priors. As in many simple linear regression problems, these priors are harmless. But are they reasonable? It is important to build reasonable priors, because as the model becomes less simple, the priors can be very helpful, but only if they are scientifically reasonable. To simulate and plot 50 prior regression lines:

```
R code
5.34 prior <- extract.prior( m5.5_draft )
xseq <- c(-2,2)
mu <- link( m5.5_draft , post=prior , data=list(N=xseq) )
```

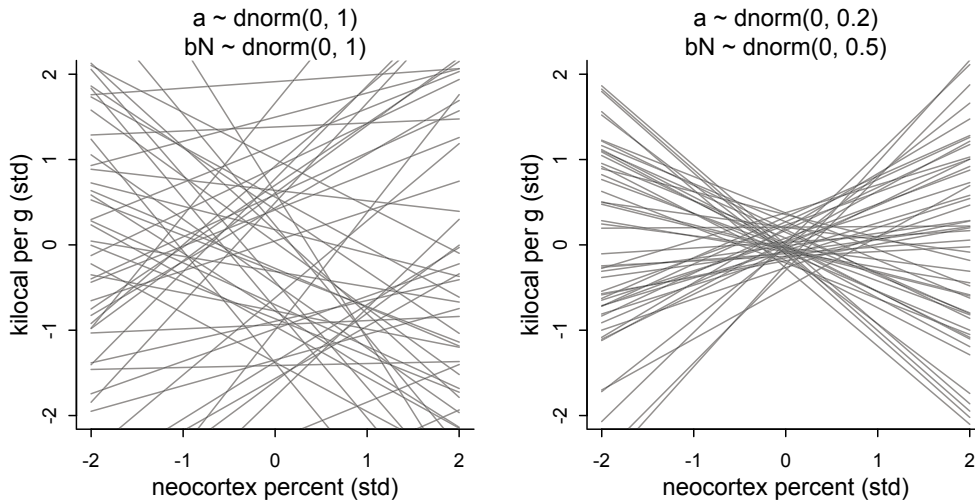



FIGURE 5.8. Prior predictive distributions for the first primate milk model, m5.5. Each plot shows a range of 2 standard deviations for each variable. Left: The vague first guess. These priors are clearly silly. Right: Slightly less silly priors that at least stay within the potential space of observations.

```
plot( NULL , xlim=xseq , ylim=xseq )
for ( i in 1:50 ) lines( xseq , mu[i,] , col=col.alpha("black",0.3) )
```

The result is displayed on the left side of [FIGURE 5.8](#). I've shown a range of 2 standard deviations for both variables. So that is most of the outcome space. These lines are crazy. As in previous examples, we can do better by both tightening the α prior so that it sticks closer to zero. With two standardized variables, when predictor is zero, the expected value of the outcome should also be zero. And the slope β_N needs to be a bit tighter as well, so that it doesn't regularly produce impossibly strong relationships. Here's an attempt:

```
m5.5 <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bN*N ,
    a ~ dnorm( 0 , 0.2 ) ,
    bN ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data=dcc )
```

R code
5.35

If you plot these priors, you'll get what is shown on the right side of [FIGURE 5.8](#). These are still very vague priors, but at least the lines stay within the high probability region of the observable data.

Now let's look at the posterior:

```
R code
5.36 precis( m5.5 )
```

```
      mean   sd  5.5% 94.5%
a      0.04 0.15 -0.21  0.29
bN     0.13 0.22 -0.22  0.49
sigma  1.00 0.16  0.74  1.26
```

From this summary, you can possibly see that this is neither a strong nor very precise association. The standard deviation is almost twice the posterior mean. But as always, it's much easier to see this if we draw a picture. Tables of numbers are golem speak, and we are not golems. We can plot the predicted mean and 89% compatibility interval for the mean to see this more easily. The code below contains no surprises. But if have extended the range of N values to consider, in `xseq`, so that the plot looks nicer.

```
R code
5.37 xseq <- seq( from=min(dcc$N)-0.15 , to=max(dcc$N)+0.15 , length.out=30 )
mu <- link( m5.5 , data=list(N=xseq) )
mu_mean <- apply(mu,2,mean)
mu_PI <- apply(mu,2,PI)
plot( K ~ N , data=dcc )
lines( xseq , mu_mean , lwd=2 )
shade( mu_PI , xseq )
```

I display this plot in the upper-left of [FIGURE 5.9](#). The posterior mean line is weakly positive, but it is highly imprecise. A lot of mildly positive and negative slopes are plausible, given this model and these data.

Now consider another predictor variable, adult female body mass, `mass` in the data frame. Let's use the logarithm of mass, `log(mass)`, as a predictor as well. Why the logarithm of mass instead of the raw mass in kilograms? It is often true that scaling measurements like body mass are related by magnitudes to other variables. Taking the log of a measure translates the measure into magnitudes. So by using the logarithm of body mass here, we're saying that we suspect that the magnitude of a mother's body mass is related to milk energy, in a linear fashion. Much later, in [Chapter 16](#), you'll see why these logarithmic relationships are almost inevitable results of the physics of organisms.

Now we construct a similar model, but consider the bivariate relationship between kilocalories and body mass. Since body mass is also standardized, we can use the same priors and stay within possible outcome values. But if you were a domain expert in growth, you could surely do better than this.

```
R code
5.38 m5.6 <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bM*M ,
    a ~ dnorm( 0 , 0.2 ) ,
    bM ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data=dcc )
precis(m5.6)
```

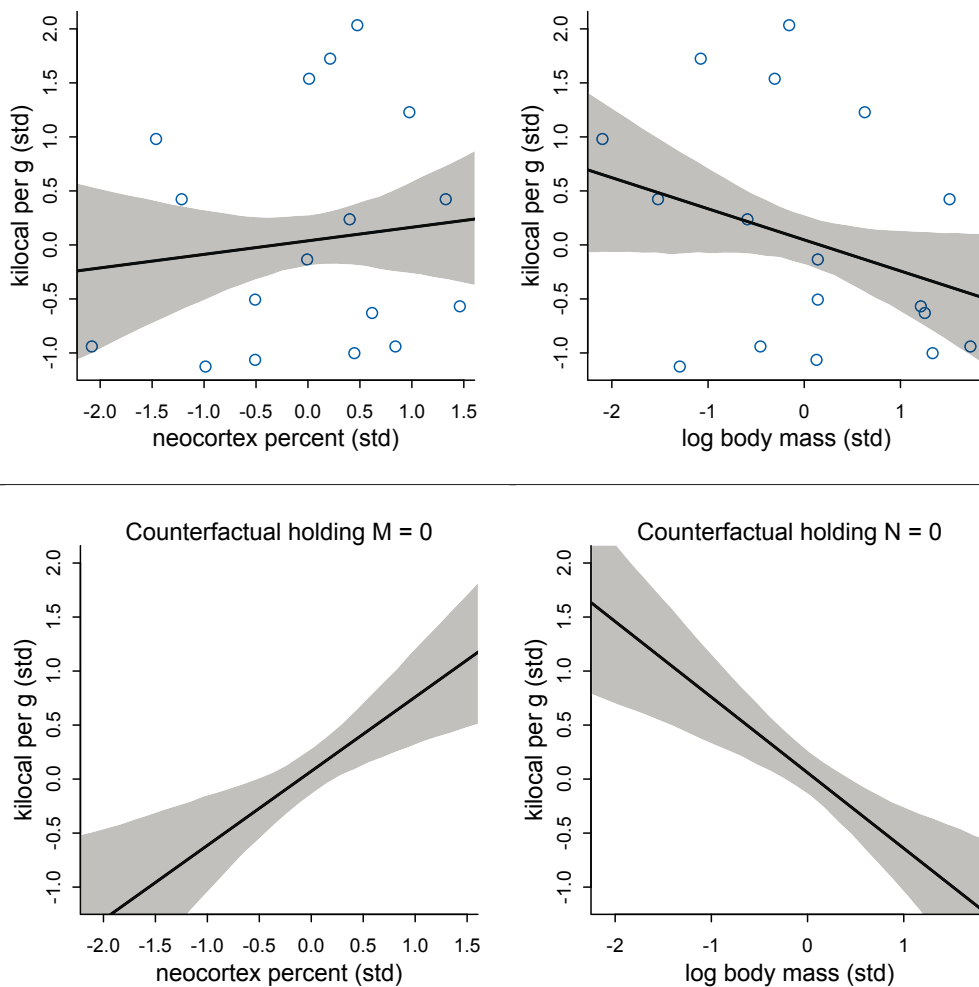


FIGURE 5.9. Milk energy and neocortex among primates. In the top two plots, simple bivariate regressions of kilocalories per gram of milk (K) on (left) neocortex percent (N) and (right) log female body mass (M) show weak associations. In the bottom row, a model with both neocortex percent (N) and log body mass (M) shows stronger associations.

	mean	sd	5.5%	94.5%
a	0.05	0.15	-0.20	0.29
bM	-0.28	0.19	-0.59	0.03
sigma	0.95	0.16	0.70	1.20

Log-mass is negatively associated with kilocalories. This association does seem stronger than that of neocortex percent, although in the opposite direction. It is quite uncertain though, with a wide compatibility interval that is consistent with a wide range of both weak and stronger relationships. This regression is shown in the upper-right of [FIGURE 5.9](#). You should modify the code that plotted the upper-left plot in the same figure, to be sure you understand how to do this.

Now let's see what happens when we add both predictor variables at the same time to the regression. This is the multivariate model, in math form:

$$\begin{aligned} K_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha + \beta_N N_i + \beta_M M_i \\ \alpha &\sim \text{Normal}(0, 0.2) \\ \beta_N &\sim \text{Normal}(0, 0.5) \\ \beta_M &\sim \text{Normal}(0, 0.5) \\ \sigma &\sim \text{Exponential}(1) \end{aligned}$$

Approximating the posterior requires no new tricks:

R code
5.39

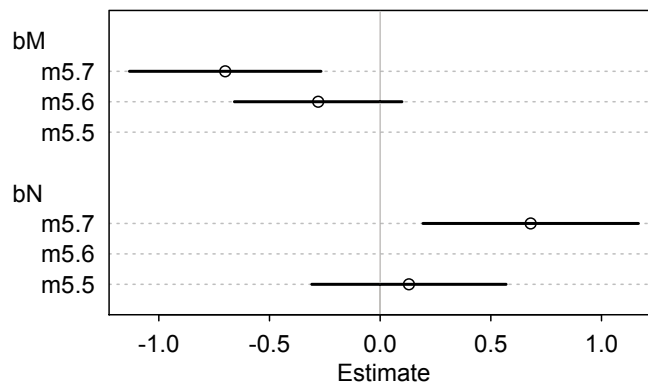
```
m5.7 <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bN*N + bM*M ,
    a ~ dnorm( 0 , 0.2 ) ,
    bN ~ dnorm( 0 , 0.5 ) ,
    bM ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data=dcc )
precis(m5.7)
```

	mean	sd	5.5%	94.5%
a	0.07	0.13	-0.15	0.28
bN	0.68	0.25	0.28	1.07
bM	-0.70	0.22	-1.06	-0.35
sigma	0.74	0.13	0.53	0.95

By incorporating both predictor variables in the regression, the posterior association of both with the outcome has increased. Visually comparing this posterior to those of the previous two models helps to see the pattern of change:

R code
5.40

```
plot( coefstab( m5.5 , m5.6 , m5.7 ) , pars=c("bM","bN") )
```

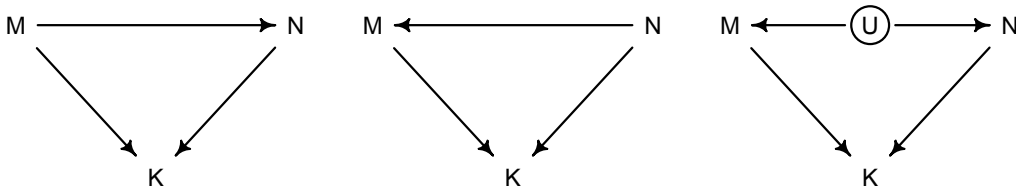


The posterior means for neocortex percent and log-mass have both moved away from zero. Adding both predictors to the model seems to have made their estimates move apart.

What happened here? Why did adding neocortex and body mass to the same model lead to stronger associations for both? This is a context in which there are two variables correlated with the outcome, but one is positively correlated with it and the other is negatively correlated with it. In addition, both of the explanatory variables are positively correlated with one another. Try a simple `pairs( ~K + M + N , dcc )` plot to appreciate this pattern of correlation. The result of this pattern is that the variables tend to cancel one another out.

This is another case in which multiple regression automatically finds the most revealing cases and uses them to produce inferences. What the regression model does is ask if species that have high neocortex percent *for their body mass* have higher milk energy. Likewise, the model asks if species with high body mass *for their neocortex percent* have higher milk energy. Bigger species, like apes, have milk with less energy. But species with more neocortex tend to have richer milk. The fact that these two variables, body size and neocortex, are correlated across species makes it hard to see these relationships, unless we account for both.

Some DAGs will help. There are at least three graphs consistent with these data.



Beginning on the left, the first possibility is that body mass (M) influences neocortex percent (N). Both then influence kilocalories in milk (K). Second, in the middle, neocortex could instead influence body mass. The two variables still end up correlated in the sample. Finally, on the right, there could be an unobserved variable U that influences both M and N , producing a correlation between them. In this book, I'll circle variables that are unobserved. One of the threats to causal inference is that there are potentially many unobserved variables that influence an outcome or the predictors. We'll consider this more in the next chapter.

Which of these graphs is right? We can't tell from the data alone, because these graphs imply the same set of **CONDITIONAL INDEPENDENCIES**. In this case, there are no conditional independencies—each DAG above implies that all pairs of variables are associated, regardless of what we condition on. A set of DAGs with the same conditional independencies is known as a **MARKOV EQUIVALENCE** set. In the Overthinking box on the next page, I'll show you how to simulate observations consistent with each of these DAGs, how each can produce the masking phenomenon, and how to use the `dagitty` package to compute the complete set of Markov equivalent DAGs. Remember that while the data alone can never tell you which causal model is correct, your scientific knowledge of the variables will eliminate a large number of silly, but Markov equivalent, DAGs.

The final thing we'd like to do with these models is to finish [FIGURE 5.9](#). Let's make *counterfactual* plots again. Suppose the third DAG above is the right one. Then imagine manipulating M and N , breaking the influence of U on each. In the real world, such experiments are impossible. If we change an animal's body size, natural selection would then change the other features to match it. But these counterfactual plots do help us see how the model views the association between each predictor and the outcome. Here is the code to produce the lower-left plot in [FIGURE 5.9](#) ([page 149](#)).

R code
5.41

```
xseq <- seq( from=min(dcc$M)-0.15 , to=max(dcc$M)+0.15 , length.out=30 )
mu <- link( m5.7 , data=data.frame( M=xseq , N=0 ) )
mu_mean <- apply(mu,2,mean)
mu_PI <- apply(mu,2,PI)
plot( NULL , xlim=range(dcc$M) , ylim=range(dcc$K) )
lines( xseq , mu_mean , lwd=2 )
shade( mu_PI , xseq )
```

You should try to reproduce the lower-right plot by modifying this code. In the practice problems, I'll ask you to consider what would happen, if you chose one of the other DAGs at the top of the page.

Overthinking: Simulating a masking relationship. Just as with understanding spurious association ([page 139](#)), it may help to simulate data in which two meaningful predictors act to mask one another. In the previous section, I showed three DAGs consistent with this. To simulate data consistent with the first DAG:

R code
5.42

```
# M -> K <- N
# M -> N
n <- 100
M <- rnorm( n )
N <- rnorm( n , M )
K <- rnorm( n , N - M )
d_sim <- data.frame(K=K,N=N,M=M)
```

You can quickly see the masking pattern of inferences by replacing `dcc` with `d_sim` in models `m5.5`, `m5.6`, and `m5.7`. Look at the `precis` summaries and you'll see the same masking pattern where the slopes become more extreme in `m5.7`. The other two DAGs can be simulated like this:

R code
5.43

```
# M -> K <- N
# N -> M
n <- 100
N <- rnorm( n )
M <- rnorm( n , N )
K <- rnorm( n , N - M )
d_sim2 <- data.frame(K=K,N=N,M=M)

# M -> K <- N
# M <- U -> N
n <- 100
U <- rnorm( n )
N <- rnorm( n , U )
M <- rnorm( n , U )
K <- rnorm( n , N - M )
d_sim3 <- data.frame(K=K,N=N,M=M)
```

In the primate milk example, it may be that the positive association between large body size and neocortex percent arises from a tradeoff between lifespan and learning. Large animals tend to live a long time. And in such animals, an investment in learning may be a better investment, because learning can be amortized over a longer lifespan. Both large body size and large neocortex then influence milk composition, but in different directions, for different reasons. This story implies that the DAG with an arrow from *M* to *N*, the first one, is the right one. But with the evidence at hand,

we cannot easily see which is right. To compute the **MARKOV EQUIVALENCE** set, let's define the first DAG and ask `dagitty` to do the hard work:

```
dag5.7 <- dagitty( "dag{
  M -> K <- N
  M -> N }" )
coordinates(dag5.7) <- list( x=c(M=0,K=1,N=2) , y=c(M=0.5,K=1,N=0.5) )
MElist <- equivalentDAGs(dag5.7)
```

R code
5.44

Now `MElist` should contain six different DAGs. To plot them all, you can use `drawdag(MElist)`. Which of these do you think you could eliminate, based upon scientific knowledge of the variables?

5.3. Categorical variables

A common question for statistical methods is to what extent an outcome changes as a result of presence or absence of a category. A category here means discrete and unordered. For example, consider the different species in the milk energy data again. Some of them are apes, while others are New World monkeys. We might want to ask how predictions should vary when the species is an ape instead of a monkey. Taxonomic group is a **CATEGORICAL VARIABLE**, because no species can be half-ape and half-monkey (discreteness), and there is no sense in which one is larger or smaller than the other (unordered). Other common examples of categorical variables include:

- Sex: male, female
- Developmental status: infant, juvenile, adult
- Geographic region: Africa, Europe, Melanesia

Many readers will already know that variables like this, routinely called **FACTORS**, can easily be included in linear models. But what is not widely understood is how these variables are represented in a model. The computer does all of the work for us, hiding the machinery. But there are some subtleties that make it worth exposing the machinery. Knowing how the machine (golem) works both helps you interpret the posterior distribution and gives you additional power in building the model.

Rethinking: Continuous countries. With automated software and lack of attention, categorical variables can be dangerous. In 2015, a high-impact journal published a study of 1170 children from six countries, finding a strong negative association between religiosity and generosity.⁸⁵ The paper caused a small stir among religion researchers, because it disagreed with the existing literature. Upon reanalysis, it was found that the country variable, which is categorical, was entered as a continuous variable instead. This made Canada (value 2) twice as much “country” as the United States (value 1). After reanalysis with country as a categorical variable, the result vanished and the original paper has been retracted. This is a happy ending, because the authors shared their data. How many cases like this exist, undiscovered because the data have never been shared and are possible lost forever?

5.3.1. Binary categories. In the simplest case, the variable of interest has only two categories, like *male* and *female*. Let's rewind to the Kalahari data you met in [Chapter 4](#). Back then, we ignored sex when predicting height, but obviously we expect males and females to have different averages. Take a look at the variables available:

R code
5.45

```
data(Howell1)
d <- Howell1
str(d)
```

```
'data.frame': 544 obs. of  4 variables:
 $ height: num  152 140 137 157 145 ...
 $ weight: num  47.8 36.5 31.9 53 41.3 ...
 $ age    : num  63 63 65 41 51 35 32 27 19 54 ...
 $ male   : int   1 0 0 1 0 1 0 1 0 1 ...
```

The male variable is our new predictor, an example of a **INDICATOR VARIABLE**. Indicator variables—sometimes also called “dummy” variables—are devices for encoding unordered categories into quantitative models. There is no sense here in which “male” is one more than “female.” The purpose of the male variable is to indicate when a person in the sample is “male.” So it takes the value 1 whenever the person is male, but it takes the value 0 when the person belongs to any other category. It doesn’t matter which category is indicated by the 1. The model won’t care. But correctly interpreting the model demands that you remember, so it’s a good idea to name the variable after the category assigned the 1 value.

There are two ways to make a model with this information. The first is to use the indicator variable directly inside the linear model, as if it were a typical predictor variable. The effect of an indicator variable is to turn a parameter on for those cases in the category. Simultaneously, the variable turns the same parameter off for those cases in another category. This will make more sense, once you see it in the mathematical definition of the model. Consider again a linear model of height, as in [Chapter 4](#). Now we’ll ignore weight and the other variables and focus only on sex.

$$h_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta_m m_i$$

$$\alpha \sim \text{Normal}(178, 20)$$

$$\beta_m \sim \text{Normal}(0, 10)$$

$$\sigma \sim \text{Uniform}(0, 50)$$

where h is height and m is the dummy variable indicating a male individual. The parameter β_m influences prediction only for those cases where $m_i = 1$. When $m_i = 0$, it has no effect on prediction, because it is multiplied by zero inside the linear model, $\alpha + \beta_m m_i$, canceling it out, whatever its value. This is just to say that, when $m_i = 1$, the linear model is $\mu_i = \alpha + \beta_m$. And when $m_i = 0$, the linear model is simply $\mu_i = \alpha$.

Using this approach means that β_m represents the expected *difference* between males and females in height. The parameter α is used to predict both female and male heights. But male height gets an extra β_m . This also means that α is no longer the average height in the sample, but rather just the average female height. This can make assigning sensible priors a little harder. If you don’t have a sense of the expected difference in height—what would be reasonable before seeing the data?—then this approach can be a bother. Of course you could get away with a vague prior in this case—there is a lot of data.

Another consequence of having to assign a prior to the difference is that this approach necessarily assumes there is more uncertainty about one of the categories—“male” in this case—than the other. Why? Because a prediction for a male includes two parameters and

therefore two priors. We can simulate this directly from the priors. The prior distributions for μ for females and males are:

```
mu_female <- rnorm(1e4,178,20)
mu_male <- rnorm(1e4,178,20) + rnorm(1e4,0,10)
precis( data.frame( mu_female , mu_male ) )
```

R code
5.46

```
'data.frame': 10000 obs. of 2 variables:
      mean    sd   5.5%  94.5% histogram
mu_female 178.41 20.04 146.30 209.94 
mu_male   177.97 22.40 142.39 214.82
```



The prior for males is wider, because it uses both parameters. While in a regression this simple, these priors will wash out very quickly, in general we should be careful. We aren't actually more unsure about male height than female height, *a priori*. Is there another way?

Another approach available to us is an **INDEX VARIABLE**. An index variable contains integers that correspond to different categories. The integers are just names, but they also let us reference a list of corresponding parameters, one for each category. In this case, we can construct our index like this:

```
d$sex <- ifelse( d$male==1 , 2 , 1 )
str( d$sex )
```

R code
5.47

```
num [1:544] 2 1 1 2 1 2 1 2 1 2 ...
```

Now “1” means female and “2” means male. No order is implied. These are just labels. And the mathematical version of the model becomes:

$$\begin{aligned}
 h_i &\sim \text{Normal}(\mu_i, \sigma) \\
 \mu_i &= \alpha_{\text{SEX}[i]} \\
 \alpha_j &\sim \text{Normal}(178, 20) \quad \text{for } j = 1..2 \\
 \sigma &\sim \text{Uniform}(0, 50)
 \end{aligned}$$

What this does is create a list of α parameters, one for each unique value in the index variable. So in this case we end up with two α parameters, named α_1 and α_2 . The numbers correspond to the values in the index variable sex. I know this seems overly complicated, but it solves our problem with the priors. Now the same prior can be assigned to each, corresponding to the notion that all the categories are the same, prior to the data. Neither category has more prior uncertainty than the other. And as you'll see in a bit, this approach extends effortlessly to contexts with more than two categories.

Let's approximate the posterior for the above model, the one using an index variable.

```
m5.8 <- quap(
  alist(
    height ~ dnorm( mu , sigma ) ,
    mu <- a[sex] ,
    a[sex] ~ dnorm( 178 , 20 ) ,
    sigma ~ dunif( 0 , 50 )
  ) , data=d )
precis( m5.8 , depth=2 )
```

R code
5.48

```

      mean   sd   5.5%  94.5%
a[1] 134.91 1.61 132.34 137.48
a[2] 142.58 1.70 139.86 145.29
sigma 27.31 0.83 25.98 28.63

```

Note the `depth=2` that I added to `precis`. This tells it to show any vector parameters, like our new `a` vector. Vector (and matrix) parameters are hidden by `precis` by default, because sometimes there are lots of these and you don't want to inspect their individual values. You'll see what I mean in later chapters.

Interpreting these parameters is easy enough—they are the expected heights in each category. But often we are interested in differences between categories. In this case, what is the expected difference between females and males? We can compute this using samples from the posterior. In fact, I'll extract posterior samples into a data frame and insert our calculation directly into the same frame:

R code
5.49

```

post <- extract.samples(m5.8)
post$diff_fm <- post$a[,1] - post$a[,2]
precis( post , depth=2 )

```

```

quap posterior: 10000 samples from m5.8
      mean   sd   5.5%  94.5%  histogram
sigma  27.29 0.84  25.95  28.63  
a[1]   134.91 1.59 132.37 137.42 
a[2]   142.60 1.71 139.90 145.35 
diff_fm -7.70 2.33 -11.41 -3.97  

```

Our calculation appears at the bottom, as a new parameter in the posterior. This is the expected difference between a female and male in the sample. This kind of calculation is called a **CONTRAST**. No matter how many categories you have, you can use samples from the posterior to compute the contrast between any two.

5.3.2. Many categories. Binary categories are easy, whether you use an indicator variable or instead an index variable. But when there are more than two categories, the indicator variable approach explodes. You'll need a new indicator variable for each new category. If you have k unique categories, you need $k - 1$ indicator variables. Automated tools like R's `lm` do in fact go this route, constructing $k - 1$ indicator variables for you and returning $k - 1$ parameters (in addition to the intercept).

But we'll instead stick with the index variable approach. It does not change at all when you add more categories. You do get more parameters, of course, just as many as in the indicator variable approach. But the model specification looks just like it does in the binary case. And the priors continue to be easier, unless you really do have prior information about contrasts. It is also important to get used to index variables, because multilevel models ([Chapter 13](#)) depend upon them.

Let's explore an example using the primate milk data again. We're interested now in the `clade` variable, which encodes the broad taxonomic membership of each species:

R code
5.50

```

data(milk)
d <- milk
levels(d$clade)

```

```
[1] "Ape" "New World Monkey" "Old World Monkey" "Strepsirrhine"
```

We want an index value for each of these four categories. You could do this by hand, but just coercing the factor to an integer will do the job:

```
d$clade_id <- as.integer( d$clade )
```

R code
5.51

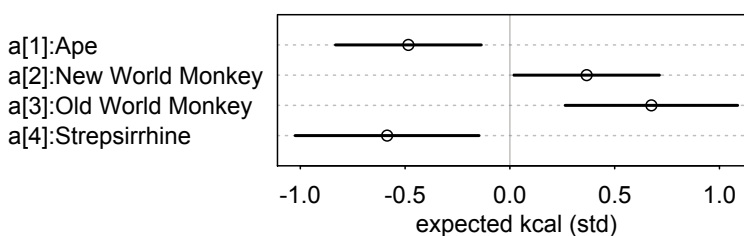
Let's use a model to measure the average milk energy in each clade. In math form:

$$\begin{aligned} K_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha_{\text{CLADE}[i]} \\ \alpha_j &\sim \text{Normal}(0, 0.5) \quad \text{for } j = 1..4 \\ \sigma &\sim \text{Exponential}(1) \end{aligned}$$

Remember, K is the standardized kilocalories. I widened the prior on α a little, to allow the different clades to disperse, if the data wants them to. But I encourage you to play with that prior and repeatedly re-approximate the posterior so you can see how the posterior differences among the categories depend upon it. Firing up quap now:

```
d$K <- standardize( d$kkal.per.g )
m5.9 <- quap(
  alist(
    K ~ dnorm( mu , sigma ),
    mu <- a[clade_id],
    a[clade_id] ~ dnorm( 0 , 0.5 ),
    sigma ~ dexp( 1 )
  ) , data=d )
labels <- paste( "a[" , 1:4 , "]:" , levels(d$clade) , sep="" )
plot( precis( m5.9 , depth=2 , pars="a" ) , labels=labels ,
      xlab="expected kcal (std)" )
```

R code
5.52



I used the optional `labels` argument to augment the parameter names `a[1]` through `a[4]` with the clade names from the original variable. In practice, you have to be very careful to keep track of which index values go with which categories. Don't trust R's factor variable type to necessarily do things right.

If you have another kind of categorical variable that you'd like to add to the model, the approach is just the same. For example, let's randomly assign these primates to some made up categories: [1] Gryffindor, [2] Hufflepuff, [3] Ravenclaw, and [4] Slytherin.

```
set.seed(63)
d$house <- sample( rep(1:4,each=8) , size=nrow(d) )
```

R code
5.53

Now we can include these categories as another predictor in the model:

```
R code
5.54 m5.10 <- quap(
      alist(
        K ~ dnorm( mu , sigma ),
        mu <- a[clade_id] + h[house],
        a[clade_id] ~ dnorm( 0 , 0.5 ),
        h[house] ~ dnorm( 0 , 0.5 ),
        sigma ~ dexp( 1 )
      ) , data=d )
```

If you inspect the posterior, you'll see that Slytherin stands out.

Rethinking: Differences and statistical significance. A common error in interpretation of parameter estimates is to suppose that because one parameter is sufficiently far from zero—is “significant”—and another parameter is not—is “not significant”—that the difference between the parameters is also significant. This is not necessarily so.⁸⁶ This isn't just an issue for non-Bayesian analysis: If you want to know the distribution of a difference, then you must compute that difference, a **CONTRAST**. It isn't enough to just observe, for example, that a slope among males overlaps a lot with zero while the same slope among females is reliably above zero. You must compute the posterior distribution of the difference in slope between males and females. For example, suppose you have posterior distributions for two parameters, β_f and β_m . β_f 's mean and standard deviation is 0.15 ± 0.02 , and β_m 's is 0.02 ± 0.10 . So while β_f is reliably different from zero (“significant”) and β_m is not, the difference between the two (assuming they are uncorrelated) is $(0.15 - 0.02) \pm \sqrt{0.02^2 + 0.1^2} \approx 0.13 \pm 0.10$. The distribution of the difference overlaps a lot with zero. In other words, you can be confident that β_f is far from zero, but you cannot be sure that the difference between β_f and β_m is far from zero.

In the context of non-Bayesian significance testing, this phenomenon arises from the fact that statistical significance is inferentially powerful in one way: difference from the null. When β_m overlaps with zero, it may also overlap with values very far from zero. Its value is uncertain. So when you then compare β_m to β_f , that comparison is also uncertain, manifesting in the width of the posterior distribution of the difference $\beta_f - \beta_m$. Lurking underneath this example is a more fundamental mistake in interpreting statistical significance: The mistake of accepting the null hypothesis. Whenever an article or book says something like “we found no difference” or “no effect,” this usually means that some parameter was not significantly different from zero, and so the authors adopted zero as the estimate. This is both illogical and extremely common.

5.4. Summary

This chapter introduced multiple regression, a way of constructing descriptive models for how the mean of a measurement is associated with more than one predictor variable. The defining question of multiple regression is: *What is the value of knowing each predictor, once we already know the other predictors?* The answer to this question does not by itself provide any causal information. Causal inference requires additional assumptions. Simple directed acyclic graph (DAG) models of causation are one way to represent those assumptions. In the next chapter we'll continue building the DAG framework and see how adding predictor variables can create as many problems as it can solve.

5.5. Practice

Problems are labeled Easy (E), Medium (M), and Hard (H).

5E1. Which of the linear models below are multiple linear regressions?

- (1) $\mu_i = \alpha + \beta x_i$
- (2) $\mu_i = \beta_x x_i + \beta_z z_i$
- (3) $\mu_i = \alpha + \beta(x_i - z_i)$
- (4) $\mu_i = \alpha + \beta_x x_i + \beta_z z_i$

5E2. Write down a multiple regression to evaluate the claim: *Animal diversity is linearly related to latitude, but only after controlling for plant diversity.* You just need to write down the model definition.

5E3. Write down a multiple regression to evaluate the claim: *Neither amount of funding nor size of laboratory is by itself a good predictor of time to PhD degree; but together these variables are both positively associated with time to degree.* Write down the model definition and indicate which side of zero each slope parameter should be on.

5E4. Suppose you have a single categorical predictor with 4 levels (unique values), labeled A, B, C and D. Let A_i be an indicator variable that is 1 where case i is in category A. Also suppose B_i , C_i , and D_i for the other categories. Now which of the following linear models are inferentially equivalent ways to include the categorical variable in a regression? Models are inferentially equivalent when it's possible to compute one posterior distribution from the posterior distribution of another model.

- (1) $\mu_i = \alpha + \beta_A A_i + \beta_B B_i + \beta_D D_i$
- (2) $\mu_i = \alpha + \beta_A A_i + \beta_B B_i + \beta_C C_i + \beta_D D_i$
- (3) $\mu_i = \alpha + \beta_B B_i + \beta_C C_i + \beta_D D_i$
- (4) $\mu_i = \alpha_A A_i + \alpha_B B_i + \alpha_C C_i + \alpha_D D_i$
- (5) $\mu_i = \alpha_A(1 - B_i - C_i - D_i) + \alpha_B B_i + \alpha_C C_i + \alpha_D D_i$

5M1. Invent your own example of a spurious correlation. An outcome variable should be correlated with both predictor variables. But when both predictors are entered in the same model, the correlation between the outcome and one of the predictors should mostly vanish (or at least be greatly reduced).

5M2. Invent your own example of a masked relationship. An outcome variable should be correlated with both predictor variables, but in opposite directions. And the two predictor variables should be correlated with one another.

5M3. It is sometimes observed that the best predictor of fire risk is the presence of firefighters—States and localities with many firefighters also have more fires. Presumably firefighters do not *cause* fires. Nevertheless, this is not a spurious correlation. Instead fires cause firefighters. Consider the same reversal of causal inference in the context of the divorce and marriage data. How might a high divorce rate cause a higher marriage rate? Can you think of a way to evaluate this relationship, using multiple regression?

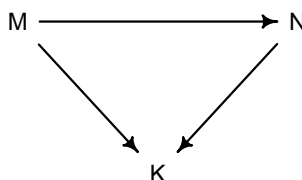
5M4. In the divorce data, States with high numbers of members of the Church of Jesus Christ of Latter-day Saints (LDS) have much lower divorce rates than the regression models expected. Find a list of LDS population by State and use those numbers as a predictor variable, predicting divorce rate using marriage rate, median age at marriage, and percent LDS population (possibly standardized). You may want to consider transformations of the raw percent LDS variable.

5M5. One way to reason through multiple causation hypotheses is to imagine detailed mechanisms through which predictor variables may influence outcomes. For example, it is sometimes argued that the price of gasoline (predictor variable) is positively associated with lower obesity rates (outcome variable). However, there are at least two important mechanisms by which the price of gas could reduce obesity. First, it could lead to less driving and therefore more exercise. Second, it could lead to less driving, which leads to less eating out, which leads to less consumption of huge restaurant meals. Can you outline one or more multiple regressions that address these two mechanisms? Assume you can have any predictor data you need.

5H1. In the divorce example, suppose the DAG is: $M \rightarrow A \rightarrow D$. What are the implied conditional independencies of the graph? Are the data consistent with it?

5H2. Assuming that the DAG for the divorce example is indeed $M \rightarrow A \rightarrow D$, fit a new model and use it to estimate the counterfactual effect of halving a State's marriage rate M . Use the counterfactual example from the chapter (starting on [page 140](#)) as a template.

5H3. Return to the milk energy model, m5.7. Suppose that the true causal relationship among the variables is:



Now compute the counterfactual effect on K of doubling M . You will need to account for both the direct and indirect paths of causation. Use the counterfactual example from the chapter (starting on [page 140](#)) as a template.

5H4. Here is an open practice problem to engage your imagination. In the divorce data, States in the southern United States have many of the highest divorce rates. Add the South indicator variable to the analysis. First, draw one or more DAGs that represent your ideas for how Southern American culture might influence any of the other three variables (D , M or A). Then list the testable implications of your DAGs, if there are any, and fit one or more models to evaluate the implications. What do you think the influence of “Southernness” is?

6 The Haunted DAG & The Causal Terror

It seems like the most newsworthy scientific studies are the least trustworthy. The more likely it is to kill you, if true, the less likely it is to be true. The more boring the topic, the more rigorous the results. How could this widely believed negative correlation exist? There doesn't seem to be any reason for studies of topics that people care about to produce less reliable results. Maybe popular topics attract more and worse researchers, like flies drawn to the smell of honey?

Actually all that is necessary for such a negative correlation to arise is that peer reviewers care about both newsworthiness and trustworthiness. Whether it is grant review or journal review, if editors and reviewers care about both, then the act of selection itself is enough to make the most newsworthy studies the least trustworthy. In fact, it's hard to imagine how scientific peer review could avoid creating this negative correlation. And, dear reader, this fact will help us understand the perils of multiple regression.

Here's a simple simulation to illustrate the point.⁸⁷ Suppose a grant review panel receives 200 research proposals. Among these proposals, there is no correlation at all between trustworthiness (rigor, scholarship, plausibility of success) and newsworthiness (social welfare value, public interest). The panel weighs trustworthiness and newsworthiness equally. Then they rank the proposals by their combined scores and select the top 10% for funding.

At the end of this section, I show the code to simulate this thought experiment. [FIGURE 6.1](#) displays the full sample of simulated proposals, with those selected in blue. I've drawn a simple linear regression line through the selected proposals. There's the negative correlation, -0.77 in this example. Strong selection induces a negative correlation among the criteria used in selection. Why? If the only way to cross the threshold is to score high, it is more common to score high on one item than on both. Therefore among funded proposals, the most newsworthy studies can actually have less than average trustworthiness (less than 0 in the figure). Similarly the most trustworthy studies can be less newsworthy than average.

This general phenomenon has been recognized for a long time. It is sometimes called [BERKSON'S PARADOX](#).⁸⁸ But it is easier to remember if we call it the *selection-distortion effect*. Once you appreciate this effect, you'll see it everywhere. Why do so many restaurants in good locations have bad food? The only way a restaurant with less-than-good food can survive is if it is in a nice location. Similarly, restaurants with excellent food can survive even in bad locations. Selection-distortion ruins your city.

What does this have to do with multiple regression? Unfortunately, everything. The previous chapter demonstrated some amazing powers of multiple regression. It can smoke out spurious correlations and clear up masking effects. This may encourage the view that, when in doubt, just add everything to the model and let the oracle of regression sort it out.

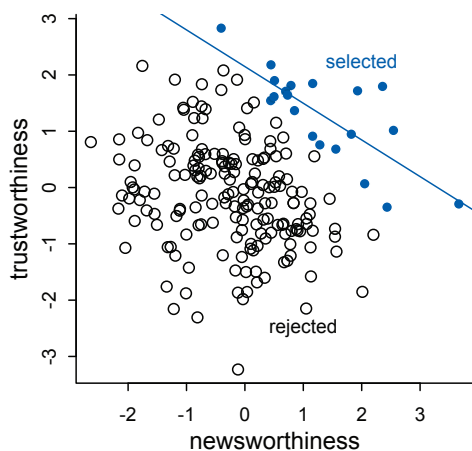


FIGURE 6.1. Why the most newsworthy studies might be the least trustworthy. 200 research proposals are ranked by combined trustworthiness and newsworthiness. The top 10% are selected for funding. While there is no correlation before selection, the two criteria are strongly negatively correlated after selection. The correlation here is -0.77 .

Regression will not sort it out. Regression is indeed an oracle, but a cruel one. It speaks in riddles and delights in punishing us for asking bad questions. The selection-distortion effect can happen inside of a multiple regression, because the act of adding a predictor induces statistical selection within the model, a phenomenon that goes by the unhelpful name **COLLIDER BIAS**. This can mislead us into believing, for example, that there is a negative association between newsworthiness and trustworthiness in general, when in fact it is just a consequence of conditioning on some variable. This is both a deeply confusing fact and one that is important to understand in order to regress responsibly.

This chapter and the next are both about terrible things that can happen when we simply add variables to a regression, without a clear idea of a causal model. In this chapter, we'll explore three different hazards: multicollinearity, post-treatment bias, and collider bias. We'll end by tying all of these examples together in a framework that can tell us which variables we must and must not add to a model in order to arrive at valid inferences. But this framework does not do the most important step for us: It will not give us a valid model.

Overthinking: Simulated science distortion. Simulations like this one are easy to do in R, or in any other scripting language, once you have seen a few examples. In this simulation, we just draw some random Gaussian criteria for a sample of proposals and then select the top 10% combined scores.

R code
6.1

```
set.seed(1914)
N <- 200 # num grant proposals
p <- 0.1 # proportion to select
# uncorrelated newsworthiness and trustworthiness
nw <- rnorm(N)
tw <- rnorm(N)
# select top 10% of combined scores
s <- nw + tw # total score
q <- quantile(s, 1-p) # top 10% threshold
selected <- ifelse(s >= q, TRUE, FALSE)
cor(tw[selected], nw[selected])
```

I chose a specific seed so you can replicate the result in [FIGURE 6.1](#), but if you rerun the simulation without the `set.seed` line, you'll see there is nothing special about the seed I used.

6.1. Multicollinearity

It is commonly true that there are many potential predictor variables to add to a regression model. In the case of the primate milk data, for example, there are 7 variables available to predict any column we choose as an outcome. Why not just build a model that includes all 7? There are several hazards.

Let's begin with the least of your worries: **MULTICOLLINEARITY**. Multicollinearity means a very strong association between two or more predictor variables. The raw correlation isn't what matters. Rather what matters is the association, conditional on the other variables in the model. The consequence of multicollinearity is that the posterior distribution will seem to suggest that none of the variables is reliably associated with the outcome, even if all of the variables are in reality strongly associated with the outcome.

This frustrating phenomenon arises from the details of how multiple regression works. In fact, there is nothing wrong with multicollinearity. The model will work fine for prediction. You will just be frustrated trying to understand it. The hope is that once you understand multicollinearity, you will better understand regression models in general.

Let's begin with a simple simulation. Then we'll turn to the primate milk data again and see multicollinearity in a real data set.

6.1.1. Multicollinear legs. Imagine trying to predict an individual's height using the length of his or her legs as predictor variables. Surely height is positively associated with leg length, or at least our simulation will assume it is. Nevertheless, once you put both legs (right and left) into the model, something vexing will happen.

The code below will simulate the heights and leg lengths of 100 individuals. For each, first a height is simulated from a Gaussian distribution. Then each individual gets a simulated proportion of height for their legs, ranging from 0.4 to 0.5. Finally, each leg is salted with a little measurement or developmental error, so the left and right legs are not exactly the same length, as is typical in real populations. At the end, the code puts height and the two leg lengths into a common data frame.

```
N <- 100                                # number of individuals
set.seed(909)
height <- rnorm(N,10,2)                  # sim total height of each
leg_prop <- runif(N,0.4,0.5)             # leg as proportion of height
leg_left <- leg_prop*height +             # sim left leg as proportion + error
  rnorm( N , 0 , 0.02 )
leg_right <- leg_prop*height +            # sim right leg as proportion + error
  rnorm( N , 0 , 0.02 )
                                           # combine into data frame
d <- data.frame(height,leg_left,leg_right)
```

R code
6.2

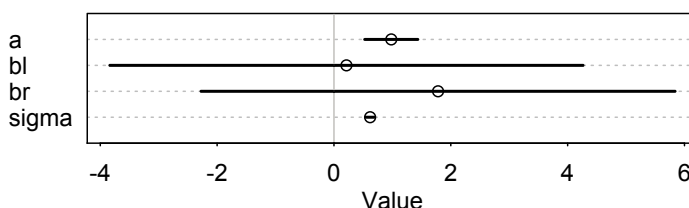
Now let's analyze these data, predicting the outcome height with both predictors, `leg_left` and `leg_right`. Before approximating the posterior, however, consider what we expect. On average, an individual's legs are 45% of their height (in these simulated data). So we should expect the beta coefficient that measures the association of a leg with height to end up around the average height (10) divided by 45% of the average height (4.5). This is $10/4.5 \approx 2.2$. Now let's see what happens instead. I'll use very vague, bad priors here, just so we can be sure that the priors aren't responsible for what is about to happen.

```
R code
6.3 m6.1 <- quap(
      alist(
        height ~ dnorm( mu , sigma ) ,
        mu <- a + bl*leg_left + br*leg_right ,
        a ~ dnorm( 10 , 100 ) ,
        bl ~ dnorm( 2 , 10 ) ,
        br ~ dnorm( 2 , 10 ) ,
        sigma ~ dexp( 1 )
      ) , data=d )
precis(m6.1)
```

	mean	sd	5.5%	94.5%
a	0.98	0.28	0.53	1.44
bl	0.21	2.53	-3.83	4.25
br	1.78	2.53	-2.26	5.83
sigma	0.62	0.04	0.55	0.69

Those posterior means and standard deviations look crazy. This is a case in which a graphical view of the `precis` output is more useful, because it displays the posterior means and 89% intervals in a way that allows us with a glance to see that something has gone wrong here:

```
R code
6.4 plot(precis(m6.1))
```



Go ahead and try the simulation a few more times, omitting the `set.seed` line. If both legs have almost identical lengths, and height is so strongly associated with leg length, then why is this posterior distribution so weird? Did the posterior approximation work correctly?

It did work correctly, and the posterior distribution here is the right answer to the question we asked. The problem is the question. Recall that a multiple linear regression answers the question: *What is the value of knowing each predictor, after already knowing all of the other predictors?* So in this case, the question becomes: *What is the value of knowing each leg's length, after already knowing the other leg's length?*

The answer to this weird question is equally weird, but perfectly logical. The posterior distribution is the answer to this question, considering every possible combination of the parameters and assigning relative plausibilities to every combination, conditional on this model and these data. It might help to look at the joint posterior distribution for `bl` and `br`:

```
R code
6.5 post <- extract.samples(m6.1)
plot( bl ~ br , post , col=col.alpha(rangi2,0.1) , pch=16 )
```

The resulting plot is shown on the left of [FIGURE 6.2](#). The posterior distribution for these two parameters is very highly correlated, with all of the plausible values of `bl` and `br` lying

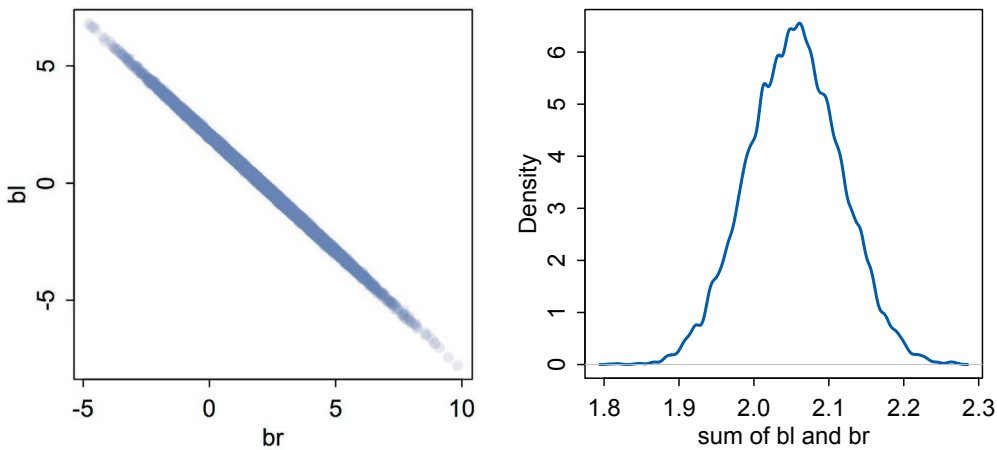


FIGURE 6.2. Left: Posterior distribution of the association of each leg with height, from model `m6.1`. Since both variables contain almost identical information, the posterior is a narrow ridge of negatively correlated values. Right: The posterior distribution of the sum of the two parameters is centered on the proper association of either leg with height.

along a narrow ridge. When `bl` is large, then `br` must be small. What has happened here is that since both leg variables contain almost exactly the same information, if you insist on including both in a model, then there will be a practically infinite number of combinations of `bl` and `br` that produce the same predictions.

One way to think of this phenomenon is that you have approximated this model:

$$y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta_1 x_i + \beta_2 x_i$$

The variable y is the outcome, like height in the example, and x is a single predictor, like the leg lengths in the example. Here x is used twice, which is a perfect example of the problem caused by using both leg lengths. From the golem's perspective, the model for μ_i is:

$$\mu_i = \alpha + (\beta_1 + \beta_2)x_i$$

All I've done is factor x_i out of each term. The parameters β_1 and β_2 cannot be pulled apart, because they never separately influence the mean μ . Only their sum, $\beta_1 + \beta_2$, influences μ . So this means the posterior distribution ends up reporting the very large range of combinations of β_1 and β_2 that make their sum close to the actual association of x with y .

And the posterior distribution in this simulated example has done exactly that: It has produced a good estimate of the sum of `bl` and `br`. Here's how you can compute the posterior distribution of their sum, and then plot it:

```
sum_blblr <- post$bl + post$br
dens( sum_blblr , col="rangi2" , lwd=2 , xlab="sum of bl and br" )
```

R code
6.6

And the resulting density plot is shown on the right-hand side of [FIGURE 6.2](#). The posterior mean is in the right neighborhood, a little over 2, and the standard deviation is much smaller

than it is for either component of the sum, `bl` or `br`. If you fit a regression with only one of the leg length variables, you'll get approximately the same posterior mean:

```
R code
6.7 m6.2 <- quap(
      alist(
        height ~ dnorm( mu , sigma ) ,
        mu <- a + bl*leg_left,
        a ~ dnorm( 10 , 100 ) ,
        bl ~ dnorm( 2 , 10 ) ,
        sigma ~ dexp( 1 )
      ) , data=d )
    precis(m6.2)
```

```
      mean   sd 5.5% 94.5%
a      1.00 0.28 0.54  1.45
bl     1.99 0.06 1.89  2.09
sigma  0.62 0.04 0.55  0.69
```

That 1.99 is almost identical to the mean value of `sum_blbr`.

The basic lesson is only this: When two predictor variables are very strongly correlated (conditional on other variables in the model), including both in a model may lead to confusion. The posterior distribution isn't wrong, in such cases. It's telling you that the question you asked cannot be answered with these data. And that's a great thing for a model to say, that it cannot answer your question. And if you are just interested in prediction, you'll find that this leg model makes fine predictions. It just doesn't make any claims about which leg is more important.

This leg example is clear and cute. But it is also purely statistical. We aren't asking any serious causal questions here. Let's try a more causally interesting example next.

6.1.2. Multicollinear milk. In the leg length example, it's easy to see that including both legs in the model is a little silly. But the problem that arises in real data sets is that we may not anticipate a clash between highly correlated predictors. And therefore we may mistakenly read the posterior distribution to say that neither predictor is important. In this section, we look at an example of this issue with real data.

Let's return to the primate milk data from earlier in the chapter:

```
R code
6.8 library(rethinking)
    data(milk)
    d <- milk
    d$K <- standardize( d$kcal.per.g )
    d$F <- standardize( d$perc.fat )
    d$L <- standardize( d$perc.lactose )
```

In this example, we are concerned with the `perc.fat` (percent fat) and `perc.lactose` (percent lactose) variables. We'll use these to model the total energy content, `kcal.per.g`. The code above has already standardized these three variables. You're going to use these three variables to explore a natural case of multicollinearity. Note that there are no missing values, NA, in these columns, so there's no need here to extract complete cases. But you can rest assured that `quap`, unlike reckless functions like `lm`, would never silently drop cases.

Start by modeling `kcal.per.g` as a function of `perc.fat` and `perc.lactose`, but in two bivariate regressions. Look back in [Chapter 5 \(page 147\)](#), for a discussion of these priors.

```
# kcal.per.g regressed on perc.fat
m6.3 <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bF*F ,
    a ~ dnorm( 0 , 0.2 ) ,
    bF ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data=d )

# kcal.per.g regressed on perc.lactose
m6.4 <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bL*L ,
    a ~ dnorm( 0 , 0.2 ) ,
    bL ~ dnorm( 0 , 0.5 ) ,
    sigma ~ dexp( 1 )
  ) , data=d )

precis( m6.3 )
precis( m6.4 )
```

R code
6.9

	mean	sd	5.5%	94.5%
a	0.00	0.08	-0.12	0.12
bF	0.86	0.08	0.73	1.00
sigma	0.45	0.06	0.36	0.54

	mean	sd	5.5%	94.5%
a	0.00	0.07	-0.11	0.11
bL	-0.90	0.07	-1.02	-0.79
sigma	0.38	0.05	0.30	0.46

The posterior distributions for `bF` and `bL` are essentially mirror images of one another. The posterior mean of `bF` is as positive as the mean of `bL` is negative. Both are narrow posterior distributions that lie almost entirely on one side or the other of zero. Given the strong association of each predictor with the outcome, we might conclude that both variables are reliable predictors of total energy in milk, across species. The more fat, the more kilocalories in the milk. The more lactose, the fewer kilocalories in milk. But watch what happens when we place both predictor variables in the same regression model:

```
m6.5 <- quap(
  alist(
    K ~ dnorm( mu , sigma ) ,
    mu <- a + bF*F + bL*L ,
    a ~ dnorm( 0 , 0.2 ) ,
```

R code
6.10

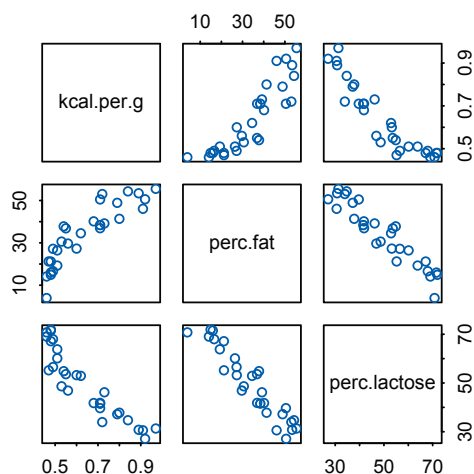


FIGURE 6.3. A pairs plot of the total energy, percent fat, and percent lactose variables from the primate milk data. Percent fat and percent lactose are strongly negatively correlated with one another, providing mostly the same information.

```

      bF ~ dnorm( 0 , 0.5 ) ,
      bL ~ dnorm( 0 , 0.5 ) ,
      sigma ~ dexp( 1 )
    ) ,
    data=d )
precis( m6.5 )

```

	mean	sd	5.5%	94.5%
a	0.00	0.07	-0.11	0.11
bF	0.24	0.18	-0.05	0.54
bL	-0.68	0.18	-0.97	-0.38
sigma	0.38	0.05	0.30	0.46

Now the posterior means of both `bF` and `bL` are closer to zero. And the standard deviations for both parameters are twice as large as in the bivariate models (`m6.3` and `m6.4`).

This is the same statistical phenomenon as in the leg length example. What has happened is that the variables `perc.fat` and `perc.lactose` contain much of the same information. They are almost substitutes for one another. As a result, when you include both in a regression, the posterior distribution ends up describing a long ridge of combinations of `bF` and `bL` that are equally plausible. In the case of the fat and lactose, these two variables form essentially a single axis of variation. The easiest way to see this is to use a pairs plot:

R code
6.11

```

pairs( ~ kcal.per.g + perc.fat + perc.lactose , data=d , col=range2 )

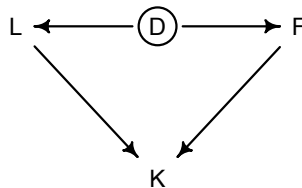
```

I display this plot in [FIGURE 6.3](#). Along the diagonal, the variables are labeled. In each scatterplot off the diagonal, the vertical axis variable is the variable labeled on the same row and the horizontal axis variable is the variable labeled in the same column. For example, the two scatterplots in the first row in [FIGURE 6.3](#) are `kcal.per.g` (vertical) against `perc.fat` (horizontal) and then `kcal.per.g` (vertical) against `perc.lactose` (horizontal). Notice

that percent fat is positively correlated with the outcome, while percent lactose is negatively correlated with it. Now look at the right-most scatterplot in the middle row. This plot is the scatter of percent fat (vertical) against percent lactose (horizontal). Notice that the points line up almost entirely along a straight line. These two variables are negatively correlated, and so strongly so that they are nearly redundant. Either helps in predicting kcal.per.g, but neither helps as much *once you already know the other*.

In the scientific literature, you might encounter a variety of dodgy ways of coping with multicollinearity. Few of them take a causal perspective. Some fields actually teach students to inspect pairwise correlations before fitting a model, to identify and drop highly correlated predictors. This is a mistake. Pairwise correlations are not the problem. It is the conditional associations—not correlations—that matter. And even then, the right thing to do will depend upon what is causing the collinearity. The associations within the data alone are not enough to decide what to do.

What is likely going on in the milk example is that there is a core tradeoff in milk composition that mammal mothers must obey. If a species nurses often, then the milk tends to be watery and low in energy. Such milk is high in sugar (lactose). If instead a species nurses rarely, in short bouts, then the milk needs to be higher in energy. Such milk is very high in fat. This implies a causal model something like this:



The central tradeoff decides how dense, D , the milk needs to be. We haven't observed this variable, so it's shown circled. Then fat, F , and lactose, L , are determined. Finally, the composition of F and L determines the kilocalories, K . If we could measure D , or had an evolutionary and economic model to predict it based upon other aspects of a species, that would be better than stumbling through regressions.

The problem of multicollinearity is a member of a family of problems with fitting models, a family sometimes known as **NON-IDENTIFIABILITY**. When a parameter is non-identifiable, it means that the structure of the data and model do not make it possible to estimate the parameter's value. Sometimes this problem arises from mistakes in coding a model, but many important types of models present non-identifiable or weakly identifiable parameters, even when coded completely correctly. Nature does not owe us easy inference, even when the model is correct.

In general, there's no guarantee that the available data contain much information about a parameter of interest. When that's true, your Bayesian machine will return a posterior distribution very similar to the prior. Comparing the posterior to the prior can therefore be a good idea, a way of seeing how much information the model extracted from the data. When the posterior and prior are similar, it doesn't mean the calculations are wrong—you got the right answer to the question you asked. But it might lead you to ask a better question.

Rethinking: Identification guaranteed; comprehension up to you. Technically speaking, *identifiability* is not a concern for Bayesian models. The reason is that as long as the posterior distribution is proper—which just means that it integrates to 1—then all of the parameters are identified. But this

technical fact doesn't also mean that you can make sense of the posterior distribution. So it's probably better to speak of *weakly identified* parameters in a Bayesian context. But the difference may be only technical. The truth is that even when a DAG says a causal effect should be identifiable, it may not be statistically identifiable. We have to work just as hard at the statistics as we do at the design.

Overthinking: Simulating collinearity. To see how imprecise of the posterior increases with association between two predictors, let's use a simulation. The code below makes a function that generates correlated predictors, fits a model, and returns the standard deviation of the posterior distribution for the slope relating `perc.fat` to `kcal.per.g`. Then the code repeatedly calls this function, with different degrees of correlation as input, and collects the results.

R code
6.12

```
library(rethinking)
data(milk)
d <- milk
sim.coll <- function( r=0.9 ) {
  d$x <- rnorm( nrow(d) , mean=r*d$perc.fat ,
    sd=sqrt( (1-r^2)*var(d$perc.fat) ) )
  m <- lm( kcal.per.g ~ perc.fat + x , data=d )
  sqrt( diag( vcov(m) ) )[2] # stddev of parameter
}
rep.sim.coll <- function( r=0.9 , n=100 ) {
  stddev <- replicate( n , sim.coll(r) )
  mean(stddev)
}
r.seq <- seq(from=0,to=0.99,by=0.01)
stddev <- sapply( r.seq , function(z) rep.sim.coll(r=z,n=100) )
plot( stddev ~ r.seq , type="l" , col=range(2) , lwd=2 , xlab="correlation" )
```

So for each correlation value in `r.seq`, the code generates 100 regressions and returns the average standard deviation from them. This code uses implicit flat priors, which are bad priors. So it does exaggerate the effect of collinear variables. When you use informative priors, the inflation in standard deviation can be much slower.

6.2. Post-treatment bias

It is routine to worry about mistaken inferences that arise from omitting predictor variables. Such mistakes are often called **OMITTED VARIABLE BIAS**, and the examples from the previous chapter illustrate it. It is much less routine to worry about mistaken inferences arising from *including* variables. But **INCLUDED VARIABLE BIAS** is real. Carefully randomized experiments can be ruined just as easily as uncontrolled observational studies. Blindly tossing variables into the causal salad is never a good idea.

Included variable bias takes several forms. The first is **POST-TREATMENT BIAS**.⁸⁹ Post-treatment bias is a risk in all types of studies. The language “post-treatment” comes in fact from thinking about experimental designs. Suppose for example that you are growing some plants in a greenhouse. You want to know the difference in growth under different anti-fungal soil treatments, because fungus on the plants tends to reduce their growth. Plants are initially seeded and sprout. Their heights are measured. Then different soil treatments are applied. Final measures are the height of the plant and the presence of fungus. There are four variables of interest here: initial height, final height, treatment, and presence of fungus.

Final height is the outcome of interest. But which of the other variables should be in the model? If your goal is to make a causal inference about the treatment, you shouldn't include the fungus, because it is a *post-treatment* effect.

Let's simulate some data, to make the example more transparent and see what exactly goes wrong when we include a post-treatment variable.

```
set.seed(71)
# number of plants
N <- 100

# simulate initial heights
h0 <- rnorm(N,10,2)

# assign treatments and simulate fungus and growth
treatment <- rep( 0:1 , each=N/2 )
fungus <- rbinom( N , size=1 , prob=0.5 - treatment*0.4 )
h1 <- h0 + rnorm(N, 5 - 3*fungus)

# compose a clean data frame
d <- data.frame( h0=h0 , h1=h1 , treatment=treatment , fungus=fungus )
precis(d)
```

R code
6.13

	mean	sd	5.5%	94.5%	histogram
h0	9.96	2.10	6.57	13.08	
h1	14.40	2.69	10.62	17.93	
treatment	0.50	0.50	0.00	1.00	
fungus	0.23	0.42	0.00	1.00	

Now you should have a data frame `d` with the simulated plant experiment data.

Rethinking: Causal inference heuristics. The danger of post-treatment bias has been known for a long time. So many scientists have been taught the heuristic that while it is risky to condition on post-treatment variables, pre-treatment variables are safe. This heuristic may lead to sensible estimates in many cases. But it is not principled. Pre-treatment variables can also create bias, as you'll see later in this chapter. There is nothing wrong, in principle, with heuristics. They are safe in the context for which they were developed. But we still need principles to know when to deploy them.

6.2.1. A prior is born. When designing the model, it helps to pretend you don't have the data generating process just above. In real research, you will not know the real data generating process. But you will have a lot of scientific information to guide model construction. So let's spend some time taking this mock analysis seriously.

We know that the plants at time $t = 1$ should be taller than at time $t = 0$, whatever scale they are measured on. So if we put the parameters on a scale of *proportion* of height at time $t = 0$, rather than on the absolute scale of the data, we can set the priors more easily. To make this simpler, let's focus right now only on the height variables, ignoring the predictor variables. We might have a linear model like:

$$h_{1,i} \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = h_{0,i} \times p$$